Verilog HDL基础知识

- 概述
- Verilog HDL的基本结构
- 数据类型及常量、变量
- 运算符
- 语句
- 语句的顺序执行与并行执行
- 用Verilog HDL设计数字电路

§1　概述

一、什么是Verilog HDL？

Verilog HDL是一种应用广泛的硬件描述性语言，是硬件设计人员和电子设计自动化（EDA）工具之间的界面。

其主要目的是用来编写设计文件，建立电子系统行为级的仿真模型。即利用计算机的巨大能力对用Verilog HDL或VHDL建模的复杂数字逻辑进行仿真，然后再自动综合以生成符合要求且在电路结构上可以实现的数字逻辑网表（Netlist），根据网表和某种工艺的器件自动生成具体电路，然后生成该工艺条件下这种具体电路的延时模型。仿真验证无误后用于制造ASIC芯片或写入EPLD和FPGA器件中。

二、Verilog HDL语言的主要特征

1、语法结构上，Verilog HDL语言与C语言有许多相似之处，并借鉴C语言的多种操作符和语法结构

2、Verilog HDL语言既包含一些高层程序设计语言的结构形式，同时也兼顾描述硬件线路连接的具体构件。

3、通过使用结构级或行为级描述可以在不同的抽象层次描述设计

三、为什么要用Verilog HDL？

•电子设计规模越来越大（普通设计已达几百万门的数量级），复杂度越来越高。有必要用高级语言来表达其功能，隐藏其具体的细节实现。

•提高逻辑设计的效率，降低设计成本，更重要的是缩短设计同期。

•可读性强，易修改。

注意：

1、HDL追求对硬件的全面描述，而将HDL描述在目标器件上实现是由EDA工具软件的综合器完成。受限于目标器件，并不是所有Verilog HDL语句均可被综合。

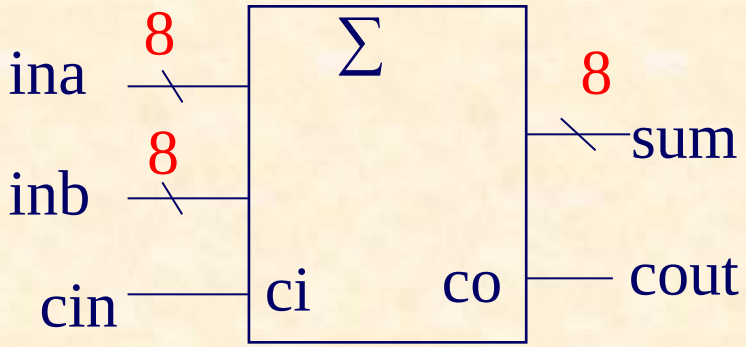
2 Verilog HDL的基本结构

2.1 简单的Verilog HDL的例子

例1 八位加法器的Verilog HDL源代码

```
module adder8 (cout, sum, ina,inb, cin);
      output[7:0]  sum;
      output  cout;
      input[7:0]   ina,inb;
      input   cin;
      assign  {cout,sum}=ina+inb+cin;
endmodule
```

准备实现的逻辑功能：



三、语句模块的描述方法

- 结构型描述
- 行为描述级
  - 系统级
  - 算法级
  - 寄存器级
- 数据流描述
- 混合型描述

（一） 结构型描述

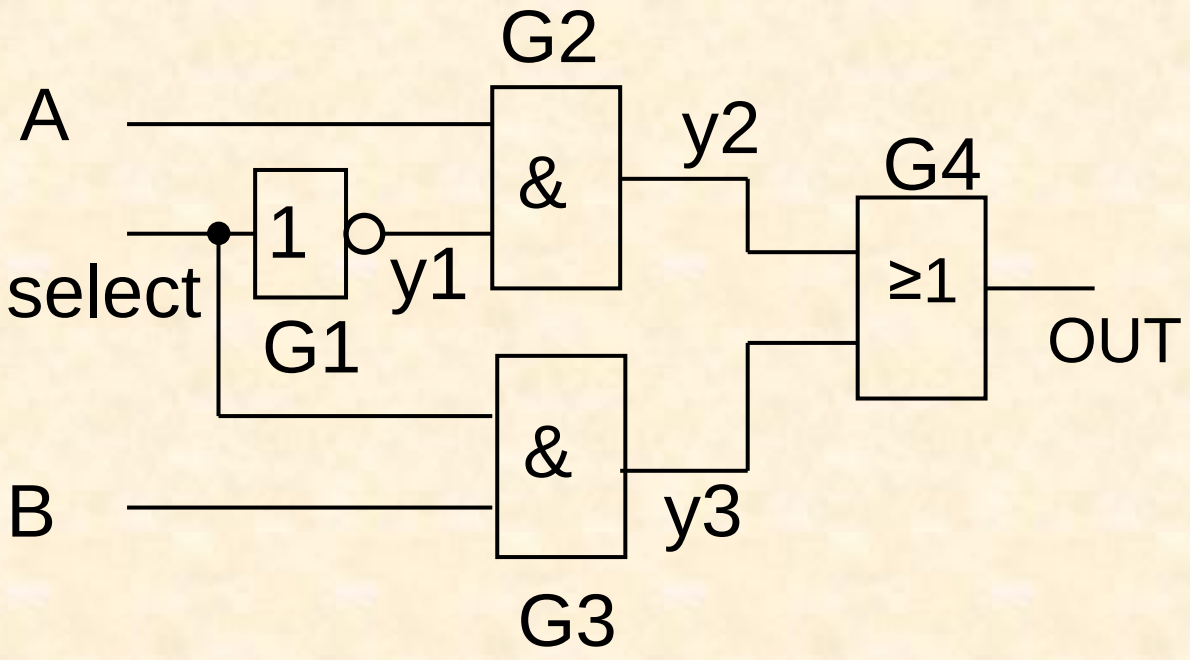
通过实例进行描述的方法，将预定义的基本元件实例嵌入到语言中，监控实例的输入，一旦其中任何一个发生变化便重新运算并输出。

1、在Verilog HDL中可使用如下结构部件：

- 在Verilog HDL中可使用如下结构部件：
- 用户自定义的模块。
- 用户自定义元件UDP。
- 内置门级元件。
- 内置开关级元件

这里讨论门级描述

2、例1： 数据选择器的结构型描述（这里为门级描述）



（1）Verilog HDL常用门的关键字

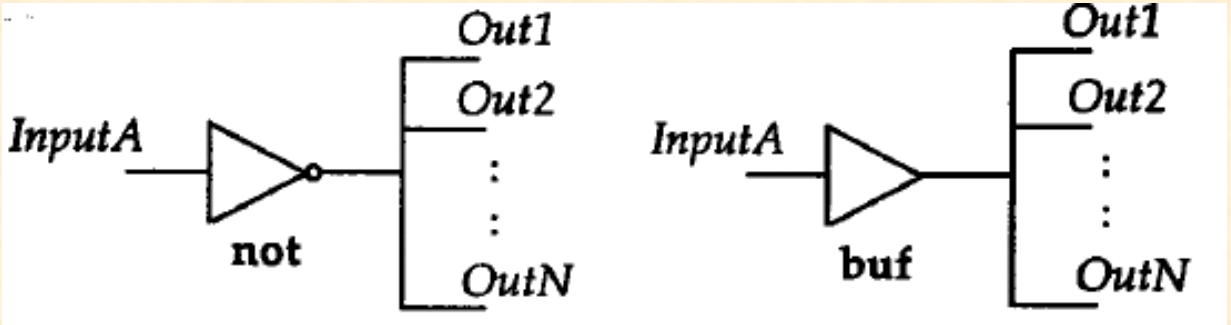
- **多输入门**

and（与门）　　nand（与非门）　　or　（或门）

nor（或非门）　　xor（异或门）　　xnor（同门）

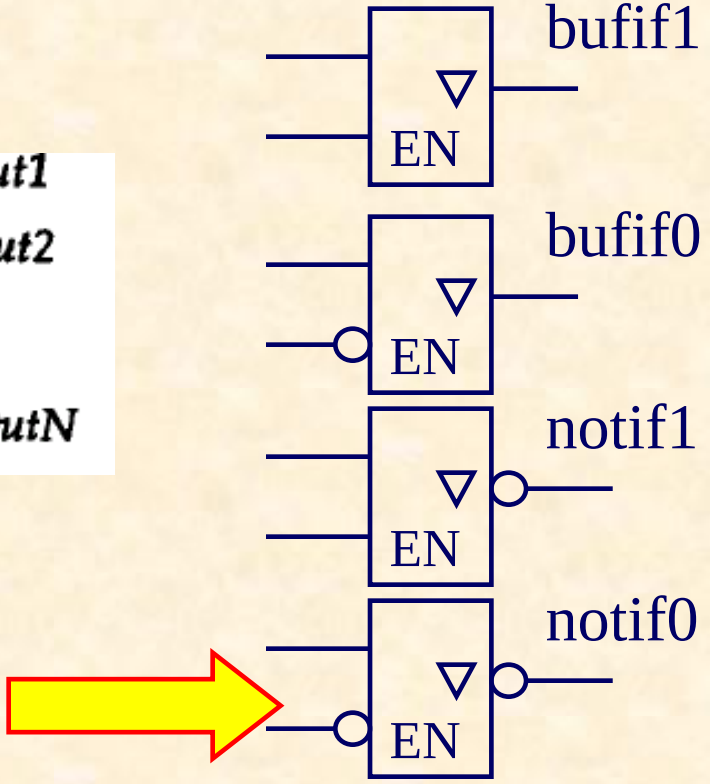
- **多输出门**

not（非门）　　buf（缓冲门）



- **三态门**

bufif1,bufif0,notif1 ,notif0(三态门)



（2） 调用门原语的句法：

语法：**门类型关键字 <例化的门名称>（端口列表）**

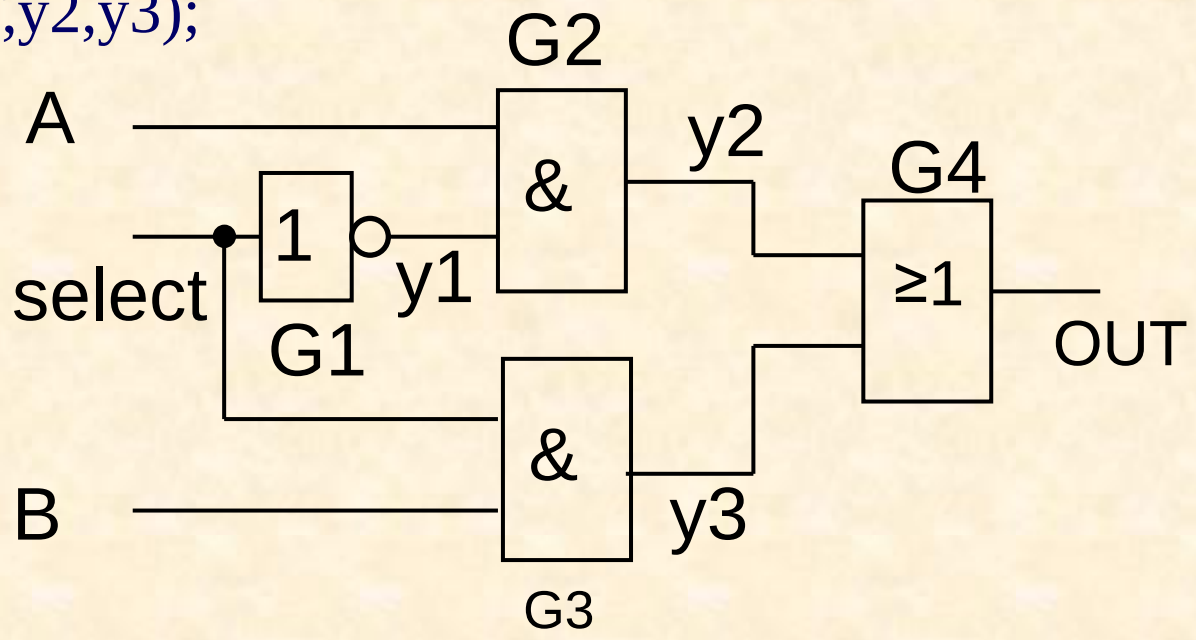
其中端口列表为：

- 多输入门：(输出，输入1，输入2，……)
- 多输出门：(输出1，输出2，……输入）
- 三态门:　　(输出，　输入，使能输入)

可以只有一个输出

（3）数据选择器的结构型描述程序清单

```
module mux2_1 (A, B, select, OUT);
        input A, B, select;              //输入、输出列表
         output OUT;
         wire  y1,y2,y3;                 //变量定义
         not G1(y1,select);               //结构描述
         and G2(y2,A,y1);
         and G3(y3,B,select);
         or  G4(OUT,y2,y3);
endmodule
```



（二） 数据流型描述

- 是一种描述组合逻辑功能的方法，用assign连续赋值语句来实现

- **连续赋值语句完成如下的组合功能：等式右边的所有变量受持续监控，每当这些变量中有任何一个发生变化，整个表达式被重新赋值并送给等式左端**

例 为数据选择器的数据流型描述。

```
module mux2x1_df (A, B, select, OUT);
   input A, B, select;
   output OUT;
   assign OUT = select ? B : A;
endmodule
```

（三） 行为级描述

always, 敏感变量发生变化，就重新一次进行赋值，有无限循环之
常用来实现时序电路，也可用来描述组合功能。

例 为数据选择器的行为型描述。

```
module mux2x1_bh(A, B, select, OUT);
        input A, B, select;
        output OUT;
        reg OUT;
        //行为型描述, select、A 、 B为敏感变量
          always @ (select or A or B)
                  if (select = = 0) OUT = A;
                  else OUT = B;
endmodule
```

（四） 混合型描述

- 用户可以混合使用上述三描述方法。
- 但需特别说明：

模块中的门的实例、模块实例语句、assign语句和always语句是并发执行的，即执行顺序跟书写次序无关。

3 数据类型及常量、变量

常用词法：

•Verilog HDL区分大小写。

•Verilog HDL的**关键字**（如：always、and、input等）都采用小写。关键字见书本449页，表9.2.3。

•Verilog HDL的注释符为：

单行注释符：//

多行注释符：/*　……　*/

3.1 常量

一.数字

- 语法：<位宽>'<进制><数值>.

四种进制表示方式：
- 二进制（b或B）
- 八进制（o或O）
- 十六进制（h或H）
- 十进制（d或D）

•4种进制表示方式：

数制	进制符号	值	例
二进制	b或B	0，1，x（不定值），z（高阻）	8'b11000101, 8'b1010zzzz, 8'b0101xxxx，7'b1010zzz
八进制	o或O	0~7，x，z	8'o305, 7'o12z
十进制	d或D	0~9	4'd61,45
十六进制	h或H	0~ f， x，z	8'hc5, 8'haz, 8‘hcx

二. 常量

语法：

parameter 参数名1=表达式，参数名2=表达式，……；

例：

parameter count_bits=8;

parameter sel=8，code=8’ha3;

parameter datawidth=8;addrwidth= datawidth*2;

使用常量的目的：（1）便于阅读，（2）便于修改。

以n位计算器为例

3.2 变量

一、分类：

- 网络型（nets type）：
  - •指硬件电路中的各种连接，
  - •输出始终根据输入的变化而更新其值的变化
- 寄存器型（register type）：
  - 常指硬件电路中具有状态保持作用的器件，如触发器、寄存器等

（一）nets型

类型	功能
wire，tri	连线类型
wor，trior	多重驱动时，具有线或功能的连线型
wand，triand	多重驱动时，具有线与功能的连线型
tri1/tri0	上拉电阻/下拉电阻
supply1/supply0	电源（逻辑1）/地（逻辑0）

以下只介绍wire型变量

1、wire型变量：

- 最常用的nets型变量
- 常用来表示用assign语句赋值的组合逻辑信号
- 取值为：0，1，x（不定值），z（高阻）
- 注意： Verilog HDL模块中的输入/输出信号类型缺省时，自动定义为wire型变量

2、wire型变量的定义

- 语法：

  wire 数据1，数据2，……数据n；

- 例子：

  wire a,b,c　　//定义了三个wire型变量a，b，c

  wire[7:0] databus　//定义了八位宽wire型向量数据总线

  wire[20:1] addrbus　//定义了20位宽wire型向量地址总线

（二）register型：

类 型	功能说明	
reg	常用的寄存器型变量	如触发器、寄存器等
integer	32位带符号整数型变量	纯数学的抽象描述，不对应任何硬件电路
real	64位带符号实数型变量	
time	无符号时间变量	

reg型变量的定义

- 语法： reg 数据1，数据2，……数据n；
- 例子：

```
reg a,b                          //定义了两个reg型变量a，b
reg[8:1]  data                   //定义了八位宽reg型向量
reg[7:0]  mymem[1023：0]         //定义1k字节（8bits）的存储器
```

二维向量称为存储器变量，MAXPLUS II不支持存储器变量

4运算符

类别	运算符	操作数个数	例子或说明
算术运算符	+ ,- ,* ,/, %(求模)	双目	17/3=5 9%4=1，
逻辑运算符	&&(与)，!(非) ||(或)	双目/单目	
位运算符	~(按位非) & (按位与) | (按位或) ^ (按位异或) ^ ~,~ ^ (按位同或)	单目 双目	若 A=5'b11001, B=5'b10101 ~A= 5'b00110 A&B= 5'b10001 A^B= 5'b01100
关系运算符	<,<=,>,>=	双目	
等式运算符	= =(相等) != (不等) = = = (全等) != = (非全等)	双目	= =与= = =区别： 例： a=5'b11x01, b= 5'b11x01则： (a= =b)=x; (a= = =b)=1

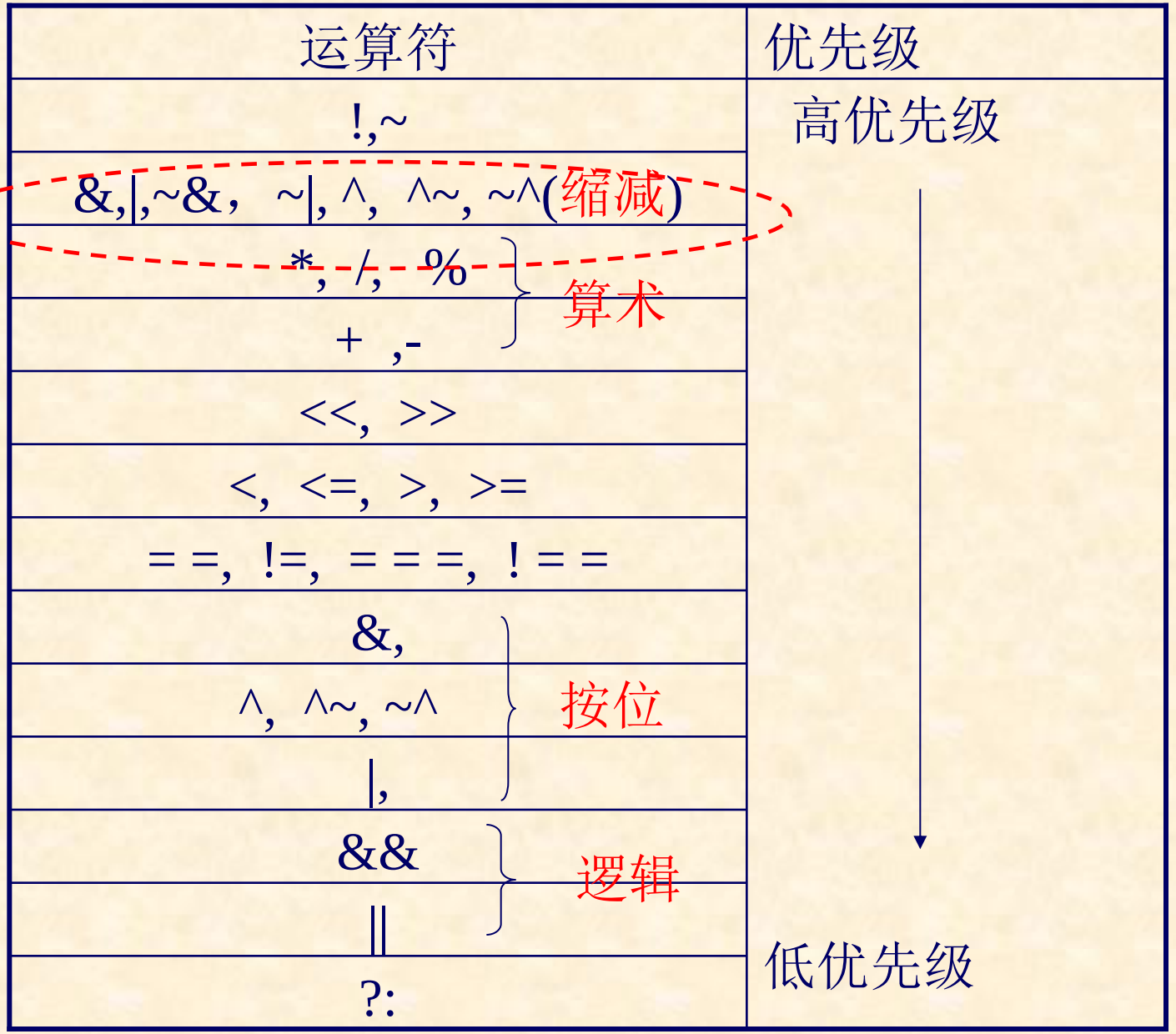
运算符(续)

与数字电路定义不同。（右移、左移互换）

类别	运算符	操作数个数	例子或说明
缩减运算符	& (缩减与) | (缩减或) ~& (缩减与非) ~| (缩减或非) ^ (缩减异或) ^ ~,~ ^ (缩减同或)	单目	若：reg[3:0] a; 则：&a=a[0]&a[1]& a[2]&a[3]
移位运算符	>>(右移) <<(左移)	单目	语法：A>>n或A<<n; 其中n为移位的位数,用0填补空位。 若 A=5'b11001,则A>>2为 5'b00110
条件运算符	?:	三目	语法：signal=condition？true_expression:false_expression; 例，2选1的MUX： out=sel?in1:in0
连接运算符	{}		{cout, sum} {a,b,c,d,e,f,g}

二、运算符的优先级：

书中有错



5 语句

一分类：

赋值语句	连续赋值语句
	过程赋值语句
条件语句	if-else语句
	case语句
循环语句	forever语句
	repeat语句
	while语句
	for语句
结构说明语句	initial语句
	always语句
	task语句
	function语句
编译预处理语句	`define语句
	`include语句
	`timescale语句

5.1 过程块

过程块：{ initial / always }

一、 always过程块

（1）always块语句模板：

```
always @(<敏感信号表达式>)
begin
    //过程赋值
    //if语句
    //case语句
    //while,repert,for 语句
    //task,functiony调用
end
```

注意：always过程块不能嵌套使用。

上升沿　　下降沿

（2）posedge与negedge关键字

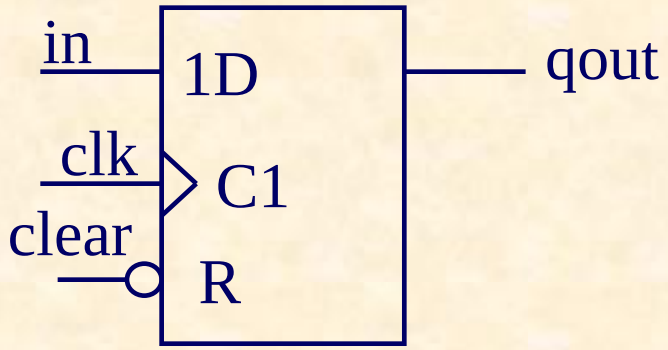
例：同步时序电路的时钟为信号为clk，clear为异步清0信号。敏感信号可写为：

```
//上升沿触发，高电平清0 有效
    always  @(posedge clk or posedge clear)
//上升沿触发，低电平清0 有效
    always  @(posedge clk or negedge clear)
```

注意：

异步控制时，块内逻辑描述要与敏感信号表达式一致

例：*带异步清0的D触发器*




```
always  @(posedge clk or negedge clear)
   begin
       if（clear）
           qout=0;
       else
           qout=in;
   end
```

矛盾？

应改为： if(!clear)

二 initial过程块

（1） initial 块语句模板：

```
initial
begin
    语句1;
    语句2;
    ……
end
```

（2）例：对变量和存贮器初始化

```
initial
begin
    reg1=0;
    for(addr=0;addr<size; addr= addr+1)
    memory[addr]=0;
end
```

(3)说明：

•主要面向功能模拟，通常不具有可综合性。

•模拟0时刻开始执行，只执行一次。

•同一模块内的多个initial过程块，模拟0时刻开始并行执行。

•initial过程块是不能嵌套使用。

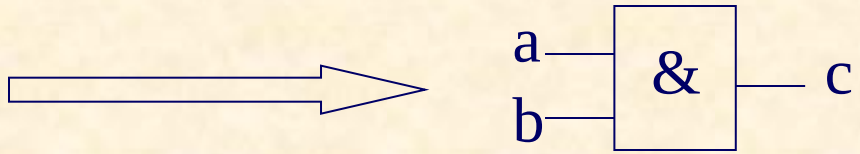
6 赋值语句

6.1 常用赋值语句

1. 连续赋值语句（assign ）:常用于对wire型变量进行赋值

例：

```
input   a，b;
output  c
assign c=a&b;
```



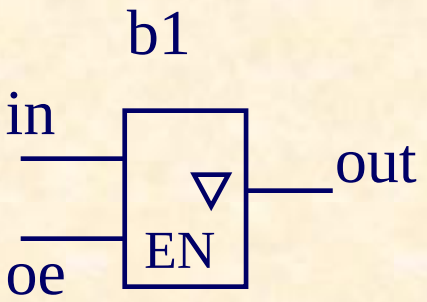
a,b信号的任何变化，都将随时反映到c上来

2. 三态门

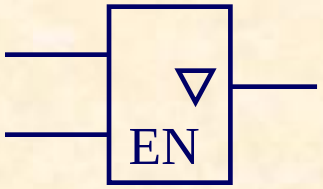
例12.2 用bufifl关键字描述的三态门

```
module tristate(in, oe, out);
        input in, oe;
        output out;
        tri out;
        //注意三态门端口的排列顺序
        bufif1 b1(out, in, oe);//(输入, 输出, 使能)
endmodule
```

与wire一样

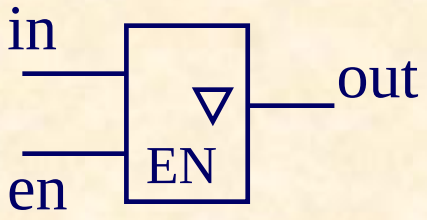


Bufif1原语：



例11.12.3 用assign描述的三态门

```
module tri_1(out, in, en);
 output  out;
 input in, en;
 assign out = en ? in : 1'bz;
//若en = 1, out = in;若en = 0 ,out为高阻态
endmodule
```



2. 过程赋值语句:常用于对reg型变量进行赋值

（1）两种方式：

非阻塞赋值:

- 一条非阻塞赋值语句的执行是**不会阻塞**下一条语句的执行，也就是说本条非阻塞赋值语句的执行完毕前，下一条语句也可开始执行。
- 非阻塞赋值语句在块结束时才完成赋值操作，在一块内非阻塞赋值语句并行执行。
- 赋值符号 <=。
- 例：b <= a

阻塞赋值:

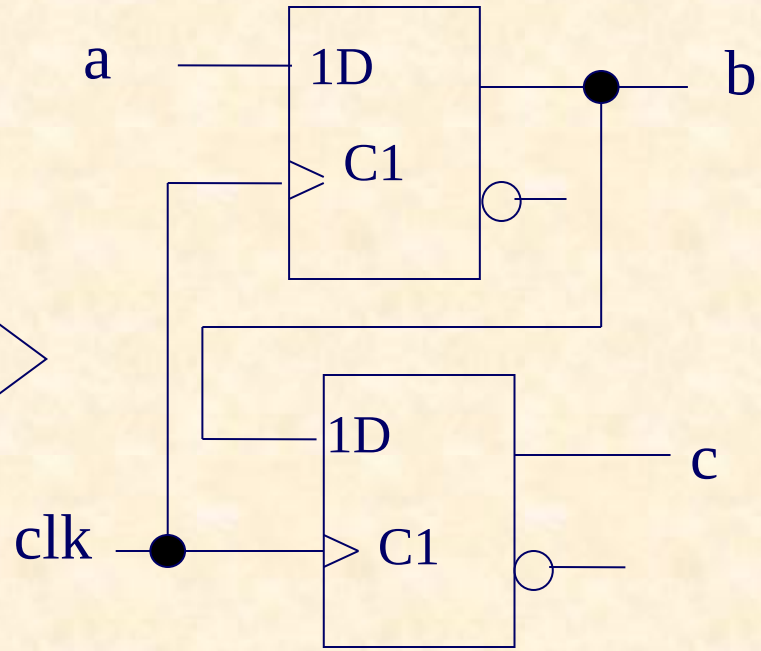
- 该语句结束时就完成赋值操作，前面的语句没有完成前，后面的语句是不能执行。
- 在一块内非阻塞赋值语句顺序执行。
- 赋值符号 =
- 例：b = a

（2）阻塞赋值和非阻塞赋值的区别：

例4 非阻塞赋值

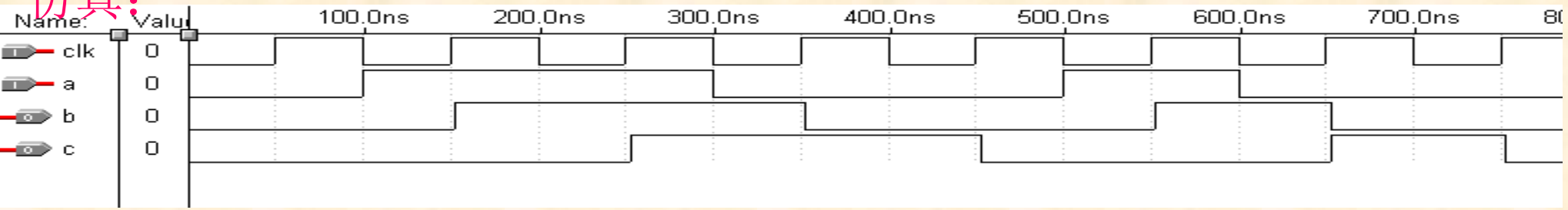
```
module non_block (c, a,b,clk);
output  c,b;
input   a,clk;
reg     c,b;
always @(posedge clk)
   begin
            b<=a;
            c<=b;
   end
endmodule
```

同时执行



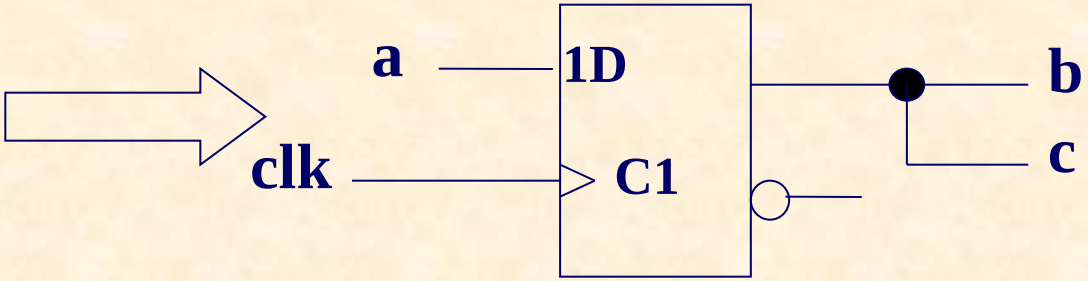
b，c在clk上沿时同时进行状态变化

仿真：



例5 阻塞赋值

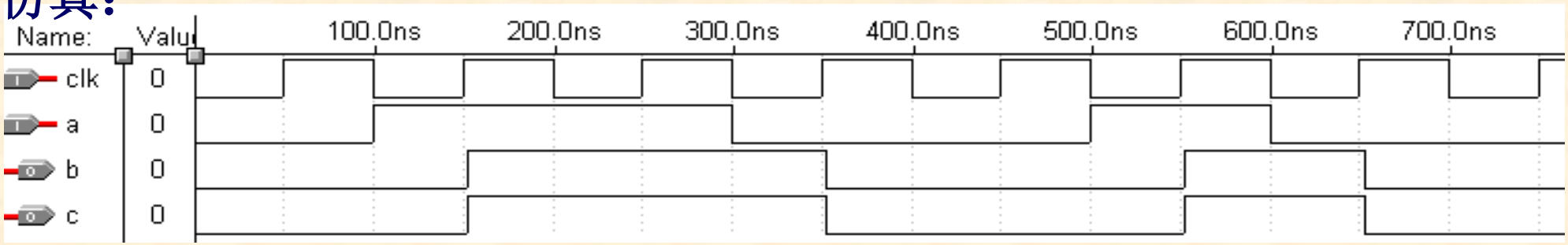
```
module   block (c, a,b,clk);
output  c,b;
input   a,clk;
reg     c,b;
always @(posedge clk)
   begin
            b=a;
            c=b;
   end
endmodule
```



顺序执行

注意： 推荐初学者使用阻塞赋值，因为类似c语言的赋值方式。

仿真：



7 条件语句

7.1 if-else语句

1. 使用形式1

if （<条件表达式>）语句或语句块；

- 条件表达式的值为1：按“真”处理；
- 条件表达式的值为0：按“假”处理。

2. 使用形式2

```
if （<条件表达式>）语句或语句块1 ；
else     语句或语句块2 ；
```

3. 使用形式3：嵌套使用

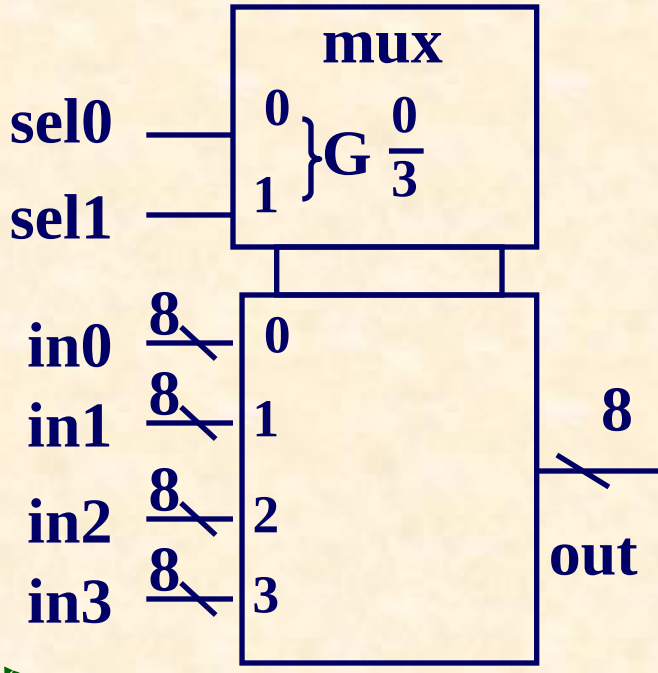
```
if （<条件表达式1>）语句或语句块1 ；
else if （<条件表达式2>）语句或语句块2 ；
……
else if （<条件表达式n>）语句或语句块n ；
else     语句或语句块n+1；
```

2. 用if-else语句描述4选1 MUX

```
module mux4_1(out , in0 , in1 , in2 , in3 , sel);
output out ;
input[7:0] in0 , in1 , in2 , in3 ;
input[1:0] sel;
reg[7:0]  out ;
always @(in0 or in1 or in2 or in3 or sel)
  begin
      if(sel== 2'b00)          out = in0;
      else if(sel == 2'b01)    out = in1;
      else if(sel == 2'b10)    out = in2;
      else                     out = in3;
  end
endmodule
```

功能描述

共有8个4选1 MUX，
它们共用sel信号



7.2 case语句

1. case语句

语法：

case （<敏感表达式>）

值1：语句或语句块1；//case分支项

值2：语句或语句块2；

……

值n：语句或语句块n；

default：语句或语句块n+1；//可省略

endcase

•当敏感表达式的值为值1时，执行语句或语句块1；为值2时，执行语句或语句块2；依此类推。

•当敏感表达式的值与所列出的值都不相等时，
条件表达式的值为0：按“假”处理。执行default后面的语句

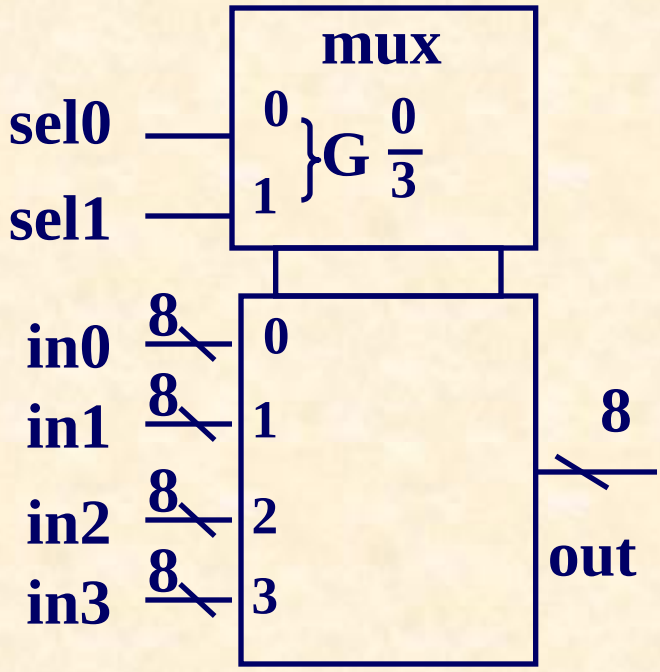
•default语句可省略。

用case语句描述*4选1 MUX*

```
module mux_4_1(out , in0 , in1 , in2 , in3 , sel);
    output[7:0] out ;
    input[7:0] in0 , in1 , in2 , in3 ;
    input[1:0] sel;
    reg[7:0] out ;
    always @(in0 or in1 or in2 or in3 or sel)
    begin
      case(sel)
         2'b00:    out = in0;
         2'b01:    out = in1;
         2'b10:    out = in2;
         default:  out = in3;
      endcase
     end
endmodule
```

真值表

共有8个4选1 MUX，
它们共用sel信号

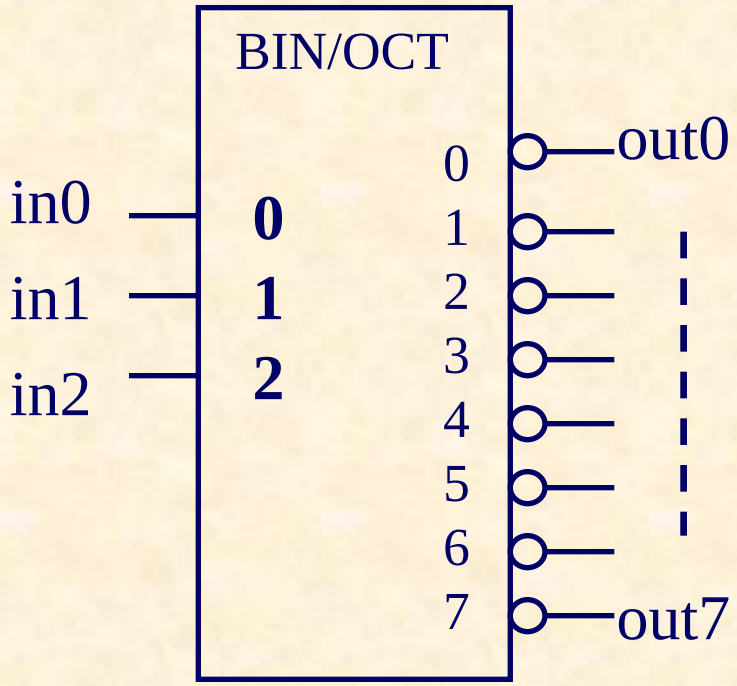


3线_8线译码器（输出低电平有效）

```
module edcoder_38(out, in);
output[7:0]  out;
input[2:0]   in;
reg[7:0]     out;
always @(in)
  begin
   case(in)
            3'd0: out = 8'b1111_1110;
            3'd1: out = 8'b1111_1101;
            3'd2: out = 8'b1111_1011;
            3'd3: out = 8'b1111_0111;
            3'd4: out = 8'b1110_1111;
            3'd5: out = 8'b1101_1111;
            3'd6: out = 8'b1011_1111;
            3'd7: out = 8'b0111_1111;
   endcase
  end
endmodule
```

分隔符，便于阅读

真值表

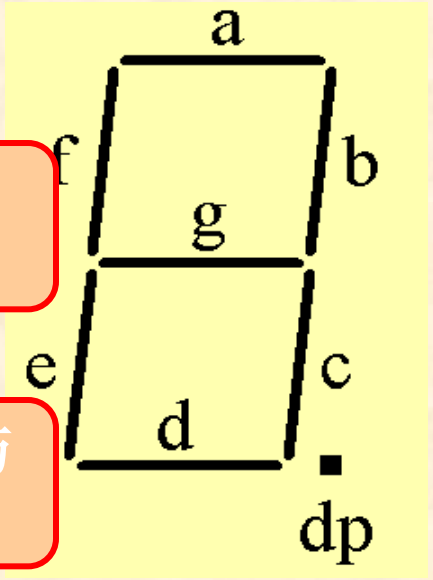


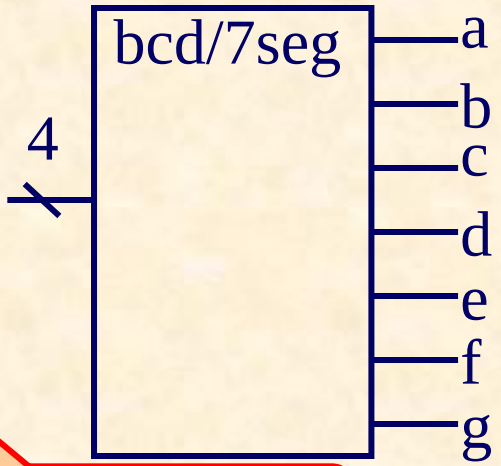
BCD码-七段 译码器(共阴)

```
module decode4_7(a,b,c,d,e,f,g,indec);
output  a,b,c,d,e,f,g;
input[3:0]   indec;
reg  a,b,c,d,e,f,g;
always  @(indec)
          begin
            case(indec)
                    4'd0   :
                    4'd1   :
                    4'd2   : {a,b,c,d,e,f,g}=7'b1101101;
                    4'd3   : {a,b,c,d,e,f,g}=7'b1111001;
                    4'd4   : {a,b,c,d,e,f,g}=7'b0110011;
                    4'd5   : {a,b,c,d,e,f,g}=7'b1011011;
                    4'd6   : {a,b,c,d,e,f,g}=7'b1011111;
                    4'd7   : {a,b,c,d,e,f,g}=7'b1110000;
                    4'd8   : {a,b,c,d,e,f,g}=7'b1111111;
                    4'd9   : {a,b,c,d,e,f,g}=7'b1111011;
                  default  : {a,b,c,d,e,f,g}=7'bx;
            endcase
          end
endmodule
```

所有输入中，任一变量变化，均执行：**组合电路**

在过程赋值，必须用**reg**，与是否组合无关





类似真值表

2.casez与casex语句

（1)语法与case语句相同

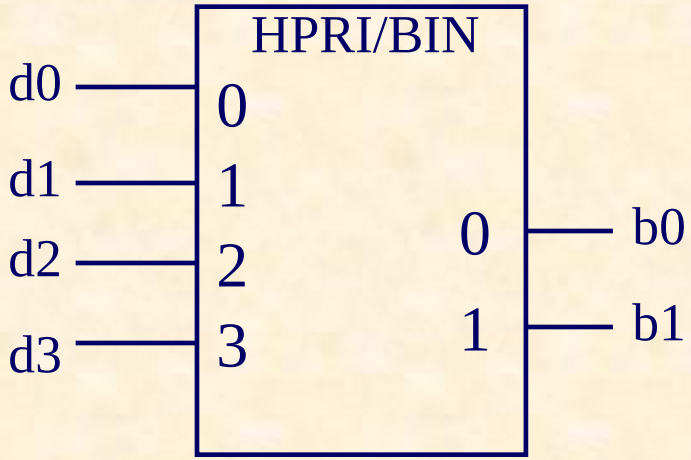
（2) casez、casex与case语句差别

•casez在语句：对分支项的值中的高阻z位不予考虑，只关心其它位的比较结果。

•casex在语句：对分支项的值中的高阻位（z）、不定值（x）均不予考虑，只关心其它位的比较结果。

•？：无关值，标识z、x另一种方法

例8 4线-2线高优先编码器



-要求：

- 输入高电平有效
- 输出二进制原码

例8 4线-2线高优先编码器

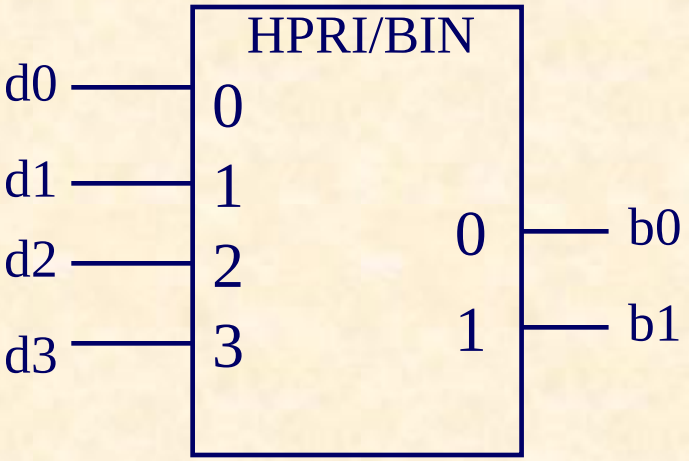
-要求：

- 输入高电平有效
- 输出二进制原码

```
module decode4_2(d,b);
output[1:0]  b;
input[3:0]   d;
reg[1:0]  b;
always  @(d)
        begin
          casex (d)
                4'b1zzx : b=2'b11;
                4'b01xx : b=2'b10;
                4'b001? : b=2'b01;
                4'b0001 : b=2'b00;
          endcase
        end
endmodule
```

应改为：
default: b=2'b00;

未定义输入0000情况，所以此程序有问题



7.3使用条件语句注意事项

注意：

- 若没有列出所有条件分支，编译器认为条件不满足时，会引进触发器保持原值。

- 时序电路可利用上述特性来保持状态。

- 组合电路必须列出所有条件分支，否则会产生隐含触发器。

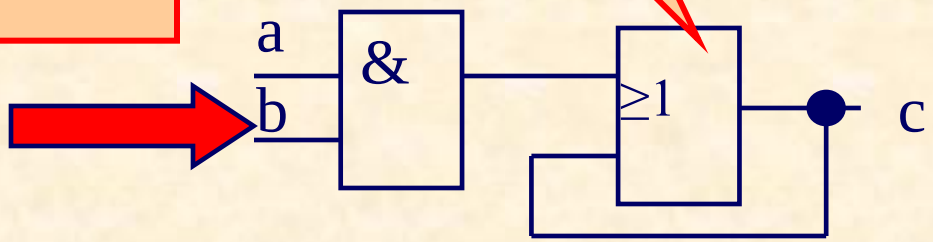
例9 隐含触发器举例:与门

```
module buried_ff(c,b,a);
output   c;
input    a,b;
reg      c;
always  @(a or b)
        begin
            if ((b==1)&&(a==1))  c=1;

        end
endmodule
```

只列出一种情况

隐含触发器



改正错误，加上
else c=0;

8 循环语句

分类

forever语句：无限循环，连续执行语句，多用initial块中，以生成周期性波形

repeat语句：连续执行语句或语句块n次

while语句：条件循环，执行语句或语句块，直到某个条件不满足

for语句：条件循环，，条件满足时，执行语句或语句块

MAX+PLUSII只支持for语句

8.1 for 语句

1.语法：

for （<变量赋初值语句>；<条件表达式>；<变量增值语句>）
　　循环体语句或语句块；

2.执行过程：

(1)执行“变量赋初值语句”

(2)判断“条件表达式”，

- 真：执行“循环体语句或语句块”,继续执行第（3）步
- 假：循环结束，退出。

(3)执行“变量增值语句”,转到（2）继续执行。

3 例子：

例10 7人投票表决器（多数表决）

```
module voter7 (pass,vote);
output  pass;
input[6:0] vote;
reg[2:0] sum;
integer i;
reg pass;
always @(vote)
  begin
    sum=0;
    for (i=0;i<=6;i=i+1)
        if(vote[i])  sum=sum+1 ;
    if (sum[2])  pass=1; //(sum>=4)==(sum[2])
    else pass=0;
  end
endmodule
```

中间变量

循环变量

for 语句

用此语句代替，更易懂。

4~7时：sum[2]=1

例11　8位二进制乘法器

```
module mult_for (outcome,a,b);
parameter   size=8 ;
output[2*size:1]  outcome;
input[size:1] a,b;  //乘数
reg[2*size:1]  outcome;  //积
integer i;
always @(a or b)
  begin
    outcome=0;
      for (i=1;i<=size;i=i+1)
      if(b[i])  outcome=outcome+( a<<(i-1)  ) ;
   end
endmodule
```

乘法方法：

为0时：左移i-1位

```
      10110111    a
    * 10010101    b
    ----------
      10110111    a
     +10110111    a左移2位
    + 10110111    a左移4位
   + 10110111     a左移7位
   ----------
```

for 语句

8.2 repeat语句

1. 语法: repeat （<循环次数表达式>） 循环体语句或语句块;

2. 例9.11 *用repeat语句实现8位乘法器*

```
module mult_repeat(outcome,a,b);
parameter   size=8 ;
output[2*size:1]  outcome;
input[size:1] a,b;
reg[2*size:1]  outcome,temp_a;
reg[size:1] temp_b;
always @(a or b)
  begin
    outcome=0;
    temp_a=a;
    temp_b=b;
    repeat(size)
      begin
       if(temp_b[1])  outcome=outcome+temp_a ;
       temp_a=temp_a<< 1;//
       temp_b=temp_b>> 1;//
       end
    end
endmodule
```

循环体语句块;

注意:
MAX+PLUSII不支持repeat语句

8.3 while和forever语句

一、 while语句

语法：

while （<条件表达式>） 循环体语句或语句块；

二、 forever语句

语法：

Forever 循环体语句或语句块；

注意：

MAX+PLUSII不支持while语句和forever语句

9 task和function语句

一、任务（task）

1.定义格式：

```
task <任务名>
        端口与类型说明;
        局部变量说明;
        语句或语句块;
endtask
```

2.调用格式：

<任务名>（端口1，端口2，……）

注意：MAX+PLUSII不支持task语句

10 编译预处理

1、 Verilog HDL语言提供了十多条的编译指令，本节只介绍常用的三种

- `define
- `include
- `timescale

2、说明

•编译指令以`（反引号）开头。

•编译指令非Verilog HDL的描述，因而始译指令结束不需要加分号。

•编译指令不受模块与文件的限制。在进行Verilog HDL语言编译时，已定义的编译指令一直有效，直至有其它编译指令修改它或取消它。

11 语句的顺序执行与并行执行

一、**assign语句之间：**并行执行（同时执行）

二、**过程块之间（ always， initial ）：并行执行**

三、 **assign语句与过程块之间：并行执行**

四、 **过程块（ always， initial ）内部**

- 串行块（begin-end）：顺序执行
- 并行块（fork-join ）：并行执行

非阻塞语句类似并行语句

注意：MAX+PLUSII不支持fork-join语句

12 用Verilog HDL设计数字电路（自学）

主要内容：

12.1 常用组合电路的设计

- 门电路的描述
- 译码器、编码器
- 数据选择器
- 奇偶校验器
- BCD码-七段 译码器
- 运算电路

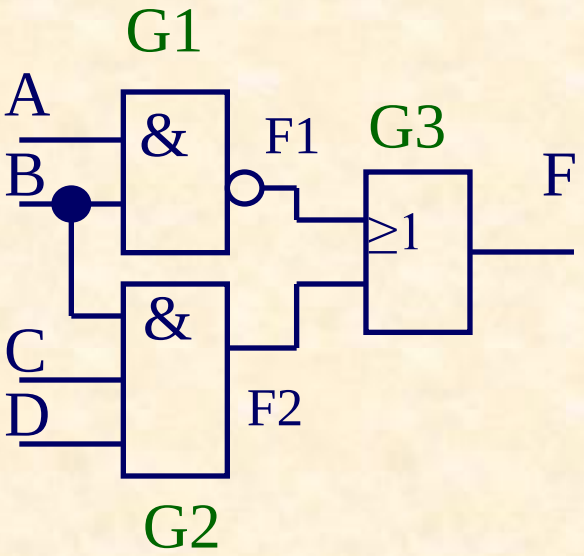
12.2常用时序电路模块

- D触发器
- 数据锁存器
- 数据寄存器
- 移位寄存器
- 各种计数器

12.1 常用组合电路的设计

12.1.1 简单门电路的描述

1.基本门电路的描述



方法1：*调用门原语*

```
module gate1(F, A, B, C, D);
   input   A, B, C, D;
   output  F;
   wire    F1,F2;
   nand    G1 ( F1,   A,  B  );
   and     G2 ( F2,   B,  C, D);
   or      G3 (  F,   F1,  F2);
endmodule
```

原语　　门的名称：可省略。

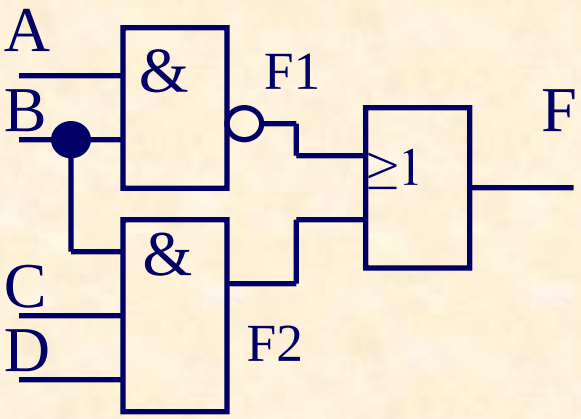
结构描述

•**调用格式：**

多输入门：原语名 < 例化名称>　（输出，输入1，输入2……）

非门：　　原语名 < 例化名称>　（输出，输入）

方法2：*用assign连续赋值语句描述*




```
module gate2(F,A,B,C,D);
  input A,B,C,D;
  output F;
  assign F=~(A & B)|(B & C & D);
endmodule
```

逻辑表达式

方法3：*用过程赋值语句描述*

```
module gate3(F,A,B,C,D);
        input A,B,C,D;
        output F;
        reg F;
        always @(A or B or C or D)
        begin
        F=~(A & B) | (B & C & D);
    end
endmodule
```

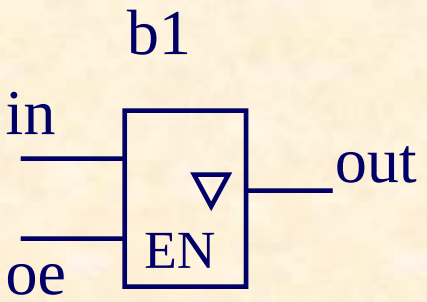
敏感信号表：
与同步时序电路差别？

2. 三态门

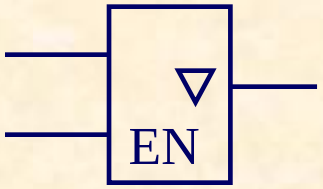
例12.2 用bufifl关键字描述的三态门

```
module tristate(in, oe, out);
        input in, oe;
        output out;
        tri out;
        //注意三态门端口的排列顺序
        bufif1 b1(out, in, oe);//(输入, 输出, 使能)
endmodule
```

与wire一样

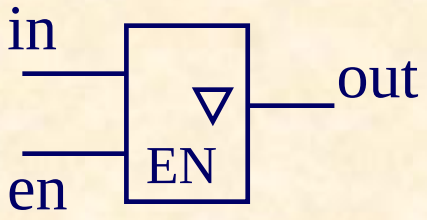


Bufif1原语：



例11.12.3 用assign描述的三态门

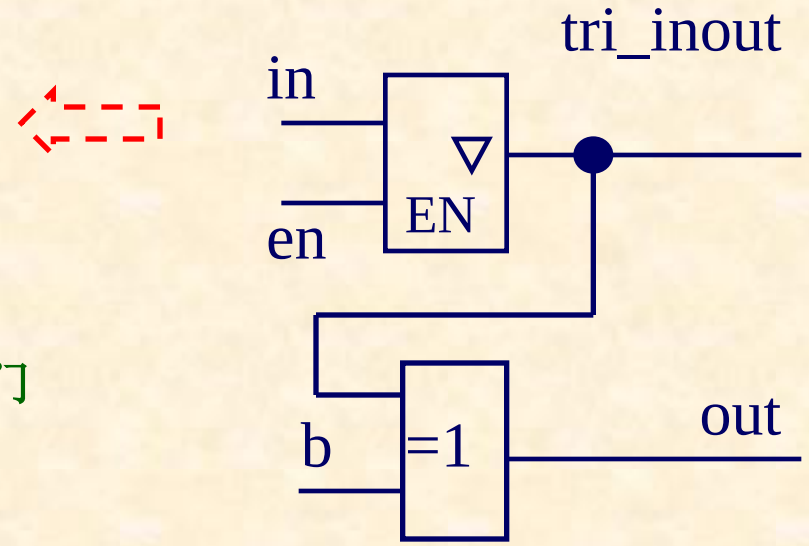
```
module tri_1(out, in, en);
 output  out;
 input in, en;
 assign out = en ? in : 1'bz;
//若en = 1, out = in;若en = 0 ,out为高阻态
endmodule
```



例12.4 用三态双向驱动器

双向端口

```
module bidir(tri_inout , out , in , en , b);
    inout tri_inout;
    output out;
    input in, en, b;
    assign tri_inout = en ? in : 1'bz;//三态门
    assign out = tri_inout ^ b;
endmodule
```



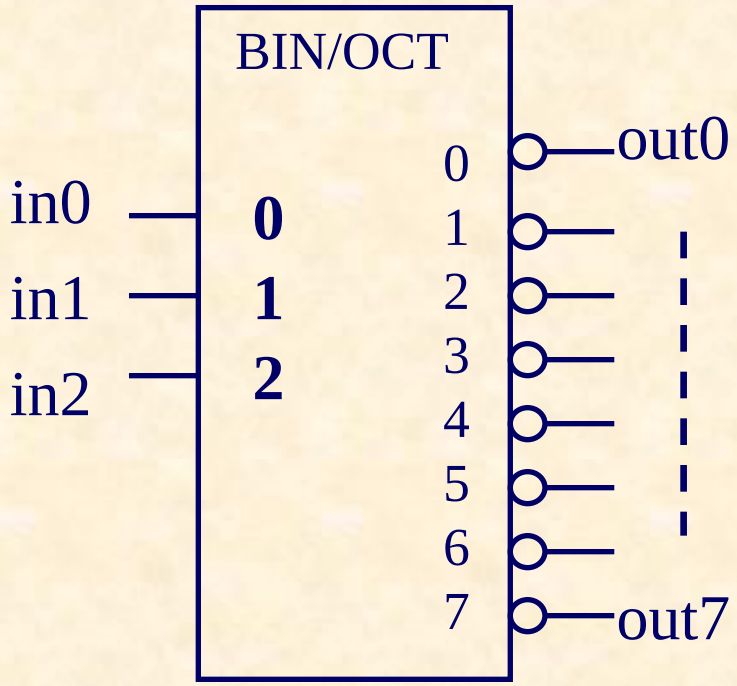
12.1.2译码器、编码器

1. 3线_8线译码器（输出低电平有效）

```
module edcoder_38(out, in);
output[7:0]  out;
input[2:0]   in;
reg[7:0]     out;
always @(in)
  begin
   case(in)
            3'd0: out = 8'b1111_1110;
            3'd1: out = 8'b1111_1101;
            3'd2: out = 8'b1111_1011;
            3'd3: out = 8'b1111_0111;
            3'd4: out = 8'b1110_1111;
            3'd5: out = 8'b1101_1111;
            3'd6: out = 8'b1011_1111;
            3'd7: out = 8'b0111_1111;
   endcase
  end
endmodule
```

分隔符，便于阅读

真值表



2. 8 线_ 3线高优先编码器

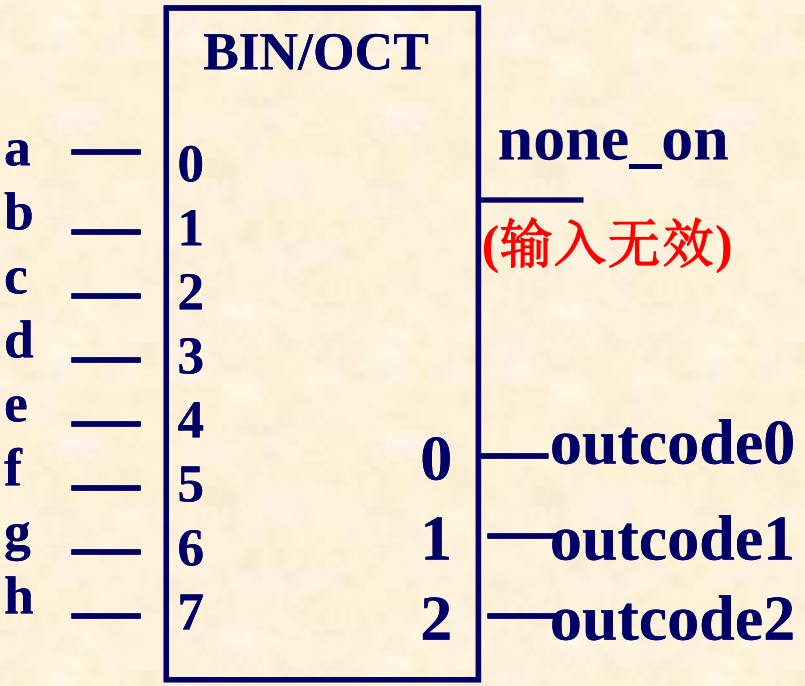
方法1：用if_else语句描述编码器

```
module encoder8_3(none_on,outcode,a, b, c, d, e, f, g, h);
output[2:0] outcode;
output      none_on;
input       a, b, c, d, e, f, g, h;
reg[3:0]    outtemp;
assign {none_on, outcode} = outtemp;
always @(a or b or c or d or e or f or g or h)
 begin
   if(h)          outtemp = 4'b0_111;
   else if(g)     outtemp = 4'b0_110;
   else if(f )    outtemp = 4'b0_101;
   else if(e)     outtemp = 4'b0_100;
   else if(d)     outtemp = 4'b0_011;
   else if(c)     outtemp = 4'b0_010;
   else if(b)     outtemp = 4'b0_001;
   else if(a)     outtemp = 4'b0_000;
   else           outtemp = 4'b1_000;
end
endmodule
```

引进变量，便于描述

none_on

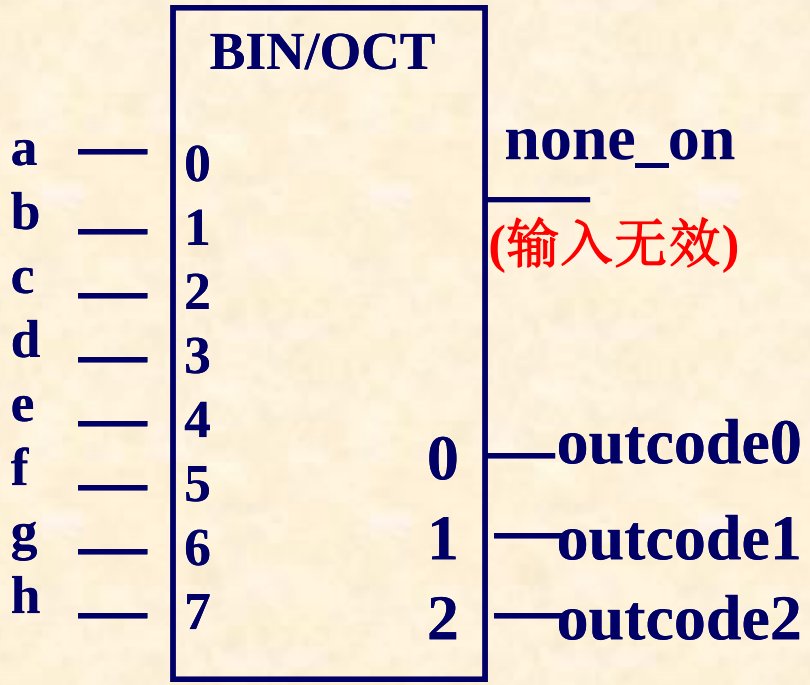
outcod



方法2：用case语句描述编码器

```
module encoder_83(none_on,outcode,a, b, c, d, e, f, g, h);
output[2:0] outcode;
output      none_on;
input       a, b, c, d, e, f, g, h;
reg[3:0]    outtemp;
assign {none_on, outcode} = outtemp;
always @(a or b or c or d or e or f or g or h)
 begin
      casex ({a, b, c, d, e, f, g, h})
          8'B????_???1   : outtemp=4'b0_111;
          8'B????_??10   : outtemp=4'b0_110;
          8'B????_?100   : outtemp=4'b0_101;
          8'B????_1000   : outtemp=4'b0_100;
          8'B???1_0000   : outtemp=4'b0_011;
          8'B??10_0000   : outtemp=4'b0_010;
          8'B?100_0000   : outtemp=4'b0_001;
          8'B1000_0000   : outtemp=4'b0_000;
          8'B0000_0000   : outtemp=4'b1_000;
      endcase
end
endmodule
```

真值表



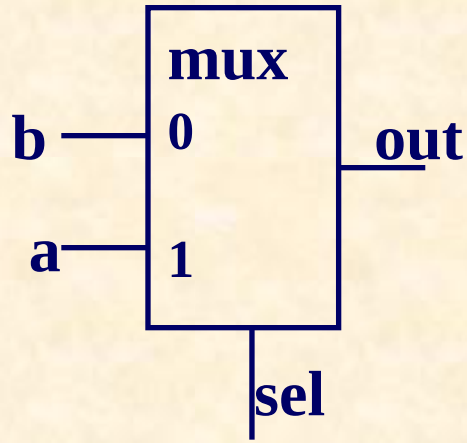
12.1.3 数据选择器

1. 用assign语句设计*2选1 MUX*

```
module  mux2_1(out , a , b , sel);
        output  out;
        input   a, b, sel;
        assign  out = sel ? a : b;
endmodule
```

sel=1：out=a

sel=0：out=b

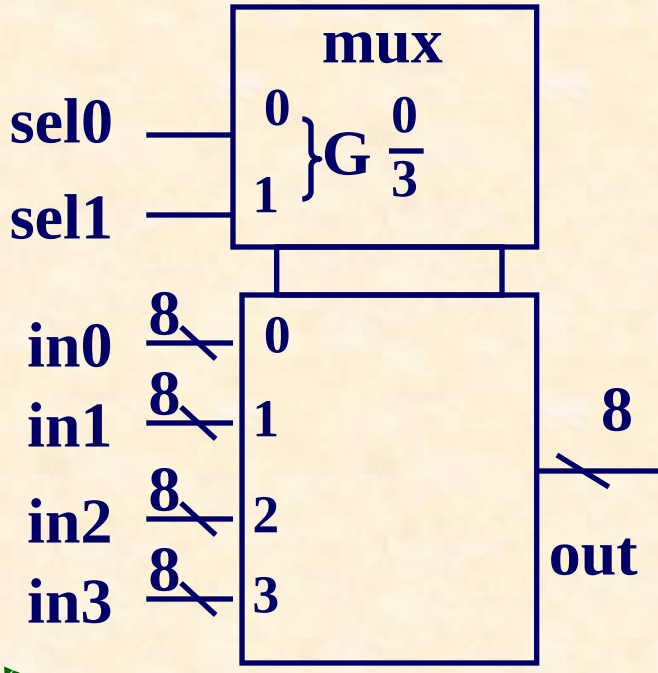


2. 用if-else语句描述4选1 MUX

```
module mux4_1(out , in0 , in1 , in2 , in3 , sel);
output out ;
input[7:0] in0 , in1 , in2 , in3 ;
input[1:0] sel;
reg[7:0]  out ;
always @(in0 or in1 or in2 or in3 or sel)
  begin
      if(sel== 2'b00)          out = in0;
      else if(sel == 2'b01)    out = in1;
      else if(sel == 2'b10)    out = in2;
      else                     out = in3;
  end
endmodule
```

功能描述

共有8个4选1 MUX，
它们共用sel信号

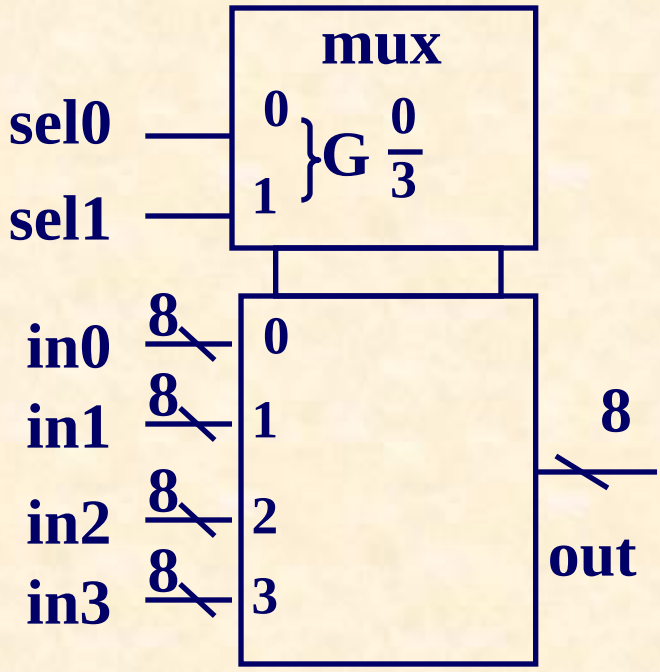


3. 用case语句描述*4选1 MUX*

```
module mux_4_1(out , in0 , in1 , in2 , in3 , sel);
    output[7:0] out ;
    input[7:0] in0 , in1 , in2 , in3 ;
    input[1:0] sel;
    reg[7:0] out ;
    always @(in0 or in1 or in2 or in3 or sel)
    begin
      case(sel)
        2'b00:    out = in0;
        2'b01:    out = in1;
        2'b10:    out = in2;
        default:  out = in3;
      endcase
    end
endmodule
```

真值表

共有8个4选1 MUX，
它们共用sel信号



12.1.4 奇偶校验器

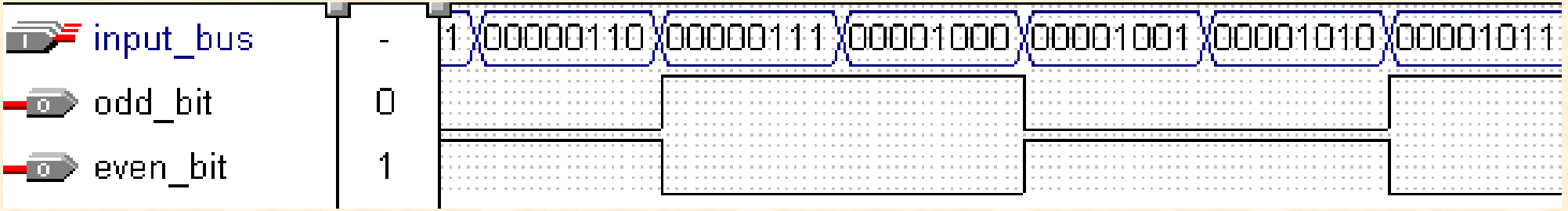
例10 奇偶位产生器

奇校验：输入数据“1”的个数为奇数。可用“异或”运算

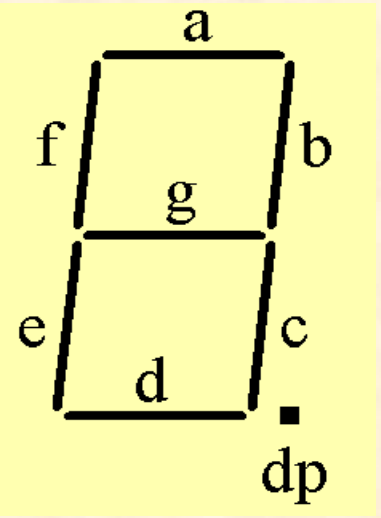
偶校验：输入数据“1”的个数为偶数。可用“同或”运算

```
module parity(even_bit ,odd_bit , input_bus);
        output      even_bit ,odd_bit ;
        input[7:0]   input_bus;
        assign odd_bit = ^ input_bus; //产生奇校验位
        assign even_bit = ~odd_bit; //产生偶校验位
endmodule
```

仿真：

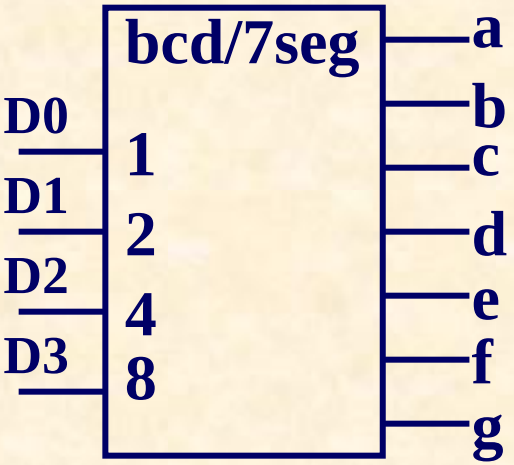


11. 12.1 .5： BCD码-七段 译码器(共阴)




```
module decode4_7(a,b,c,d,e,f,g,D);
output  a,b,c,d,e,f,g;
input[3:0]  D;
reg  a,b,c,d,e,f,g;
always  @(D)
        begin
          case(D)
                4'd0   : {a,b,c,d,e,f,g}=7'b1111110;
                4'd1   : {a,b,c,d,e,f,g}=7'b0110000;
                4'd2   : {a,b,c,d,e,f,g}=7'b1101101;
                4'd3   : {a,b,c,d,e,f,g}=7'b1111001;
                4'd4   : {a,b,c,d,e,f,g}=7'b0110011;
                4'd5   : {a,b,c,d,e,f,g}=7'b1011011;
                4'd6   : {a,b,c,d,e,f,g}=7'b1011111;
                4'd7   : {a,b,c,d,e,f,g}=7'b1110000;
                4'd8   : {a,b,c,d,e,f,g}=7'b1111111;
                4'd9   : {a,b,c,d,e,f,g}=7'b1111011;
              default  : {a,b,c,d,e,f,g}=7'bx;
          endcase
        end
endmodule
```

真值表

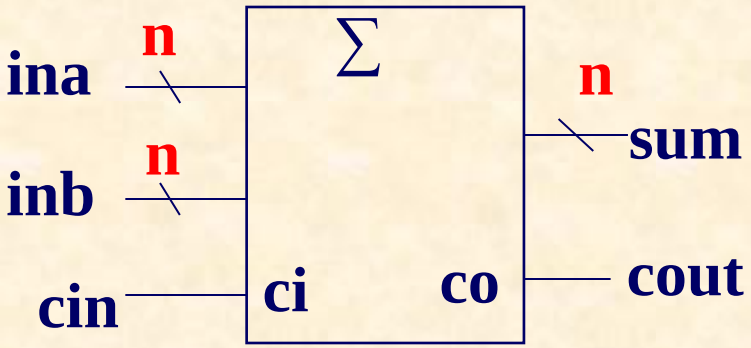


11.12.1.6 运算电路

一、加法器：

例：带进位的n位通用加法器

逻辑图：

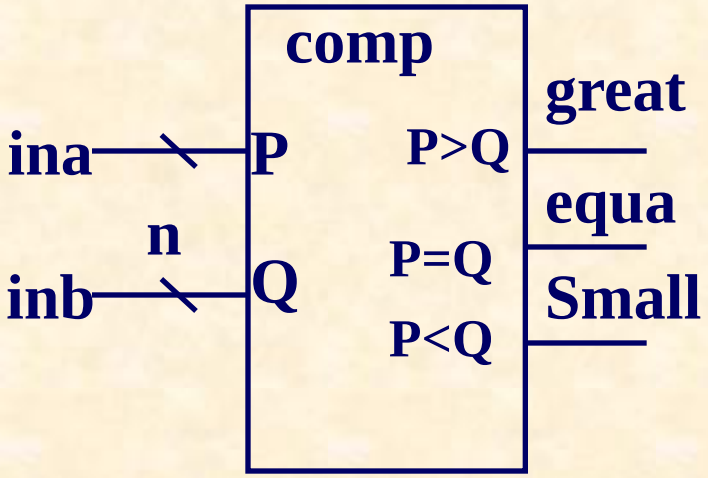



```
module adder_n (cout, sum, ina,inb, cin);
        parameter  n=16;
        output[n:1]  sum;
        output  cout;
        input[n:1]   ina,inb;
        input   cin;
        assign  {cout,sum}=ina+inb+cin;
endmodule
```

二、比较器

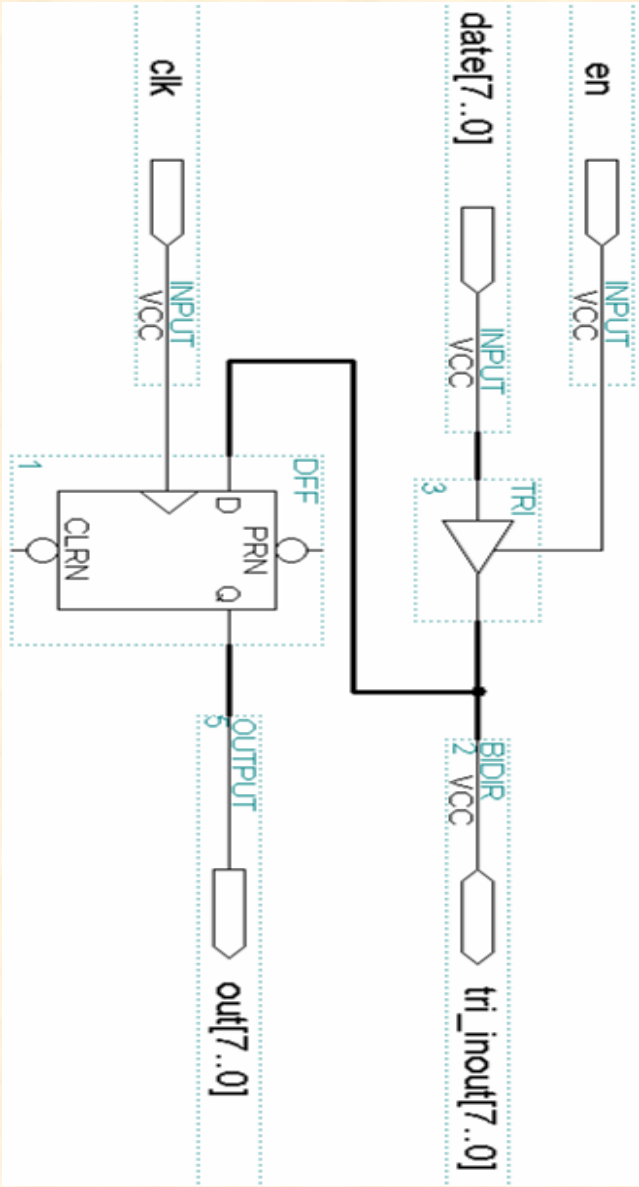
例：n位比较器

```
module compare_n (great, equal, Small,ina,inb);
        parameter  n=16;
        output  great, equal, Small;
        input[n:1]   ina,inb;
        assign  great=(ina>inb);
        assign  equal=(ina==inb);
       assign  Small=(ina<inb);
endmodule
```



12.1.7、双向三态端口的描述

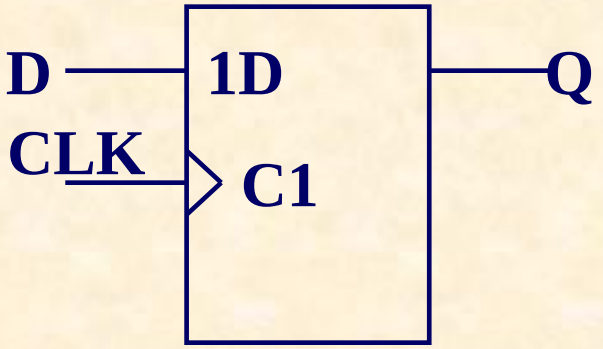
```
module tri_inout(tri_inout,out,data,en,clk);
      input          en,clk;
      input [7:0]    data;
      output[7:0]    out;
      inout [7:0]    tri_inout;
      wire  [7:0]    tri_inout;
      reg   [7:0]    out;
      assign tri_inout= en?data:8'bz;
      always @(posedge clk)
            out=tri_inout;
endmodule
```



12.2 常用时序电路模块的设计

12.2.1 D触发器

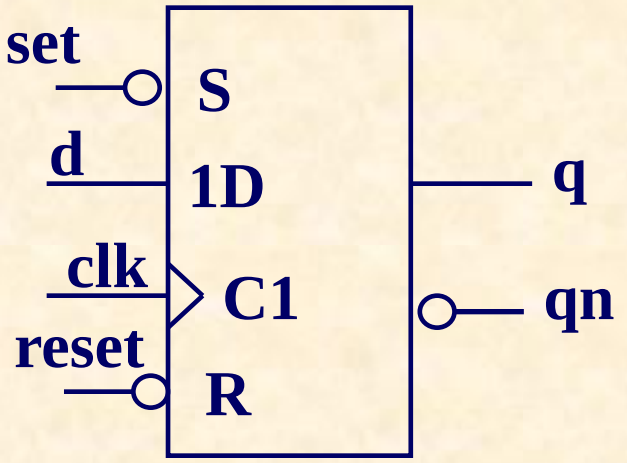
1.最简单的D触发器




```
module DFF(Q , D , CLK);
output Q;
input D , CLK;
reg Q;
always@(posedge CLK)
  begin
     Q =D;
  end
endmodule
```

不需要同步输入端D。与组合差别？

2. 带异步清0、异步置1的D触发器

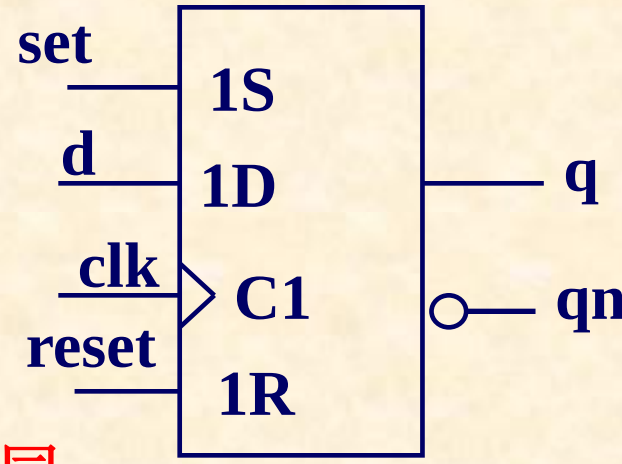



```
module DFF1(q,qn,d,clk,set,reset);
input d, clk, set, reset;
output q, qn;
reg q, qn;
always @(posedge clk or  negedge set or negedge reset)
begin
  if (! reset)
     begin  q =0; qn = 1; end   //异步清0，低电平有效
  else if (! set )
     begin  q=1; qn=0; end     //异步置1，低电平有效
       else      begin    q =d;   qn = ~d;  end
end
endmodule
```

不需要同步输入端D。与组合差别？

注意：与数字电路中的D有差别？set、reset可同时有效，但reset信号有优权。

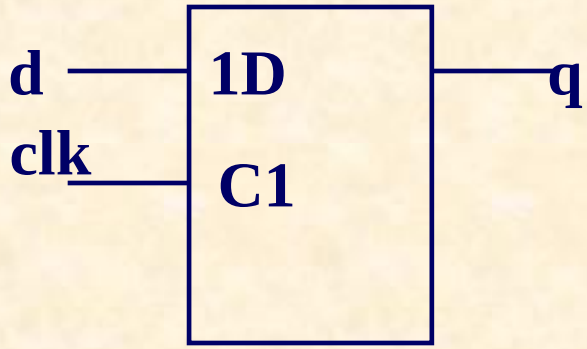
3. 带同步清0、同步置1的D触发器




```
module DFF2(q, qn, d, clk, set, reset);
input d, clk, set, reset;
output q, qn;
reg q, qn;
always @ (posedge clk)
 begin
  if(reset)
    begin   q =0; qn =1; end //同步清0，高电平有效
  else if (set)
    begin   q =1;qn =0; end  //同步置1，高电平有效
       else
        begin q = d; qn = ~d; end
 end
endmodule
```

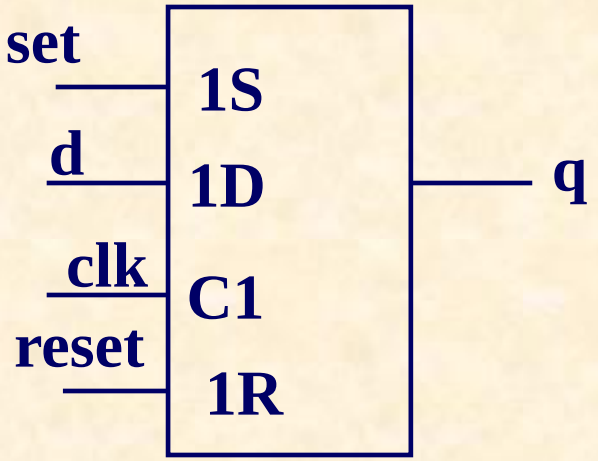
注意：清0和置1，同步与异步差别？

12.2.2 数据锁存器（latch）



1.电平敏感的一位数据锁存器(同步D)

```
module latch_1(q, d, clk);
output q;
input d, clk;
  assign q =clk ? d : q;  //时钟高电平时，将输入端数据锁存
endmodule
```

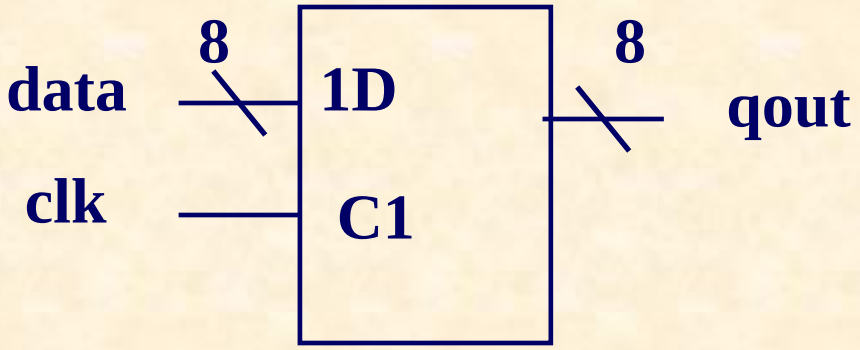


2. 带置位和复位端的1位数据锁存器

```
module latch_2(q, d, clk, set, reset);
output q;
input  d, clk, set, reset;
assign q =reset ? 0 : (set ? 1 :(clk ? d : q) );
endmodule
```

3. 8位数据锁存器

```
module latch_8(qout, data, clk);
output[7:0]   qout;
input[7:0]    data;
input         clk;
reg[7:0]      qout;
always @ (clk or data)
 begin
   if(clk)
   qout =data;
end
endmodule
```

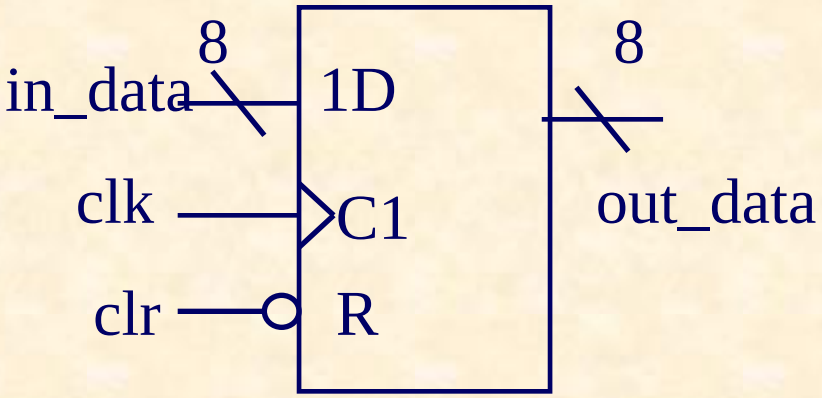


clk高电平有效，
当clk=1时：输出、输入是透明。

12.2.3 数据寄存器

例. 8位数据寄存器

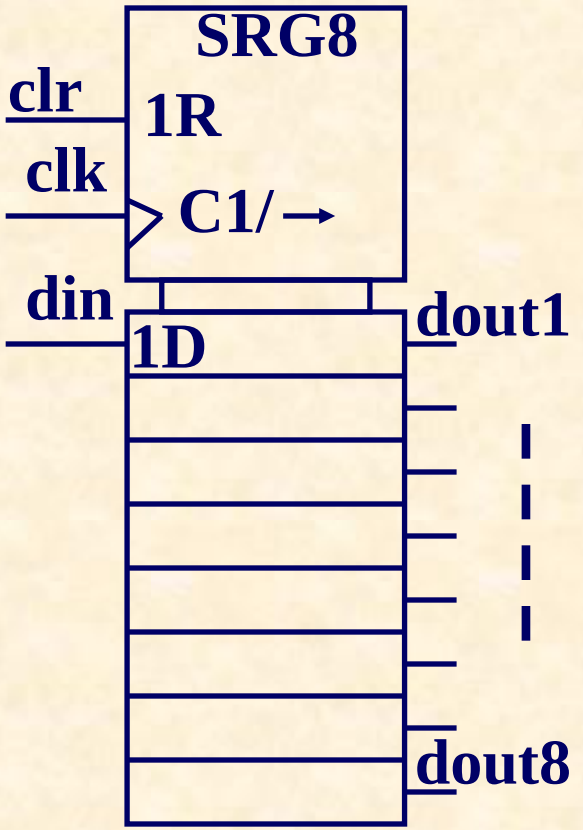
```
module reg8(out_data,in_data,clk,clr);
output[7:0]  out_data;
input        clk,clr;
input[7:0]   in_data;
reg[7:0]     out_data;
always @ (posedge clk or posedge clr)
  begin
   if(clr)    out_data =0;
   else      out_data = in_data;
 end
endmodule
```



*数据锁存器*与*数据寄存器*的差别？

电平触发　　边沿触发

12.2.4 移位寄存器（单向）




```
module shifter(din , clk , clr ,dout);
parameter n=8;
input din , clk , clr;
output[8:1] dout ;
reg[8:1] dout;
always @(posedge clk)
 begin
        if (clr) dout = 0;   // 同步清0，高电平有效
        else
         begin
           dout = dout <<  1;//输出信号左移一位
           dout[1] = din;     //输入信号补充到输出信号的最低位
         end
 end
endmodule
```

注意：左移定义

Verilog HDL：低位移向高位

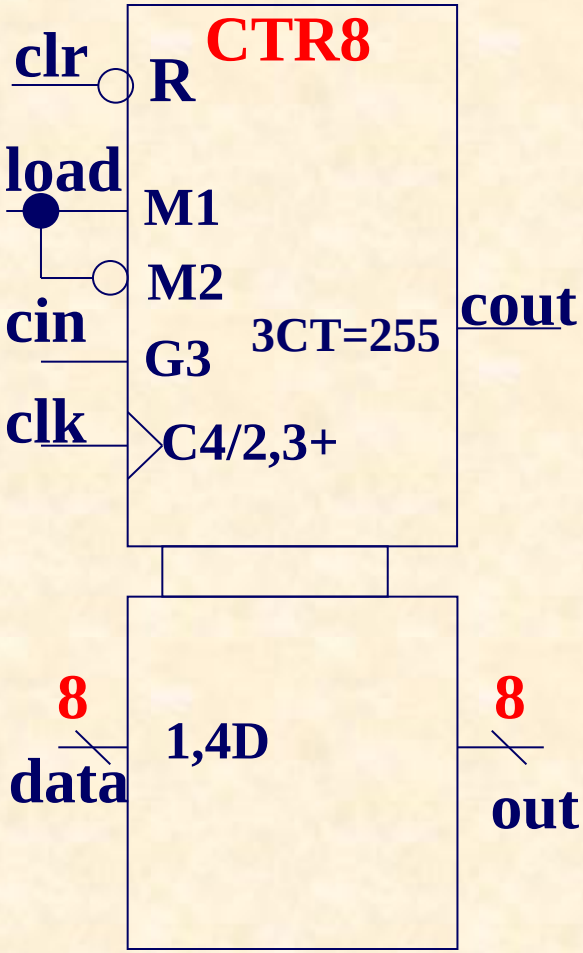
数字电路：　　高位移向低位

12.2.5 计数器

一、可预置的n位二进制计数器（带异步清0）

```
module counter_n (out, cout, data, load, cin ,clr, clk);
  parameter  n=8;
  output   [n:1] out;
  output   cout;
  input   load, cin, clr, clk;
  input  [n:1] data;
  reg     [n:1] out;
  always @(posedge clk or  negedge clr) //异步清0
    begin
       if(!clr)   out=0;
         else  if   (load)    out= data;//置数
                 else
                     out = out + cin;  //计数或保持
    end
   assign cout=&out&cin; // 进位
endmodule
```

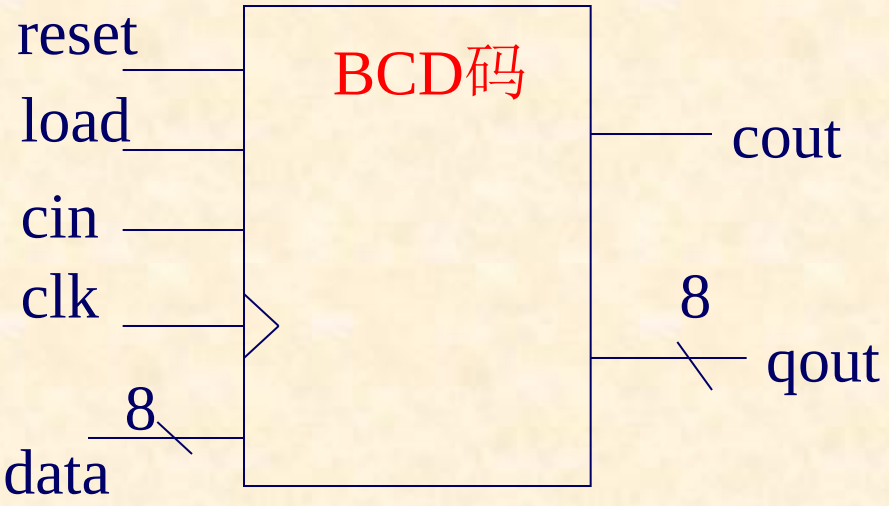
计数器的位数



例6 模为60的BCD码同步计数器

要求：

（1）输入、输出



（2）功能

reset	load	cin	clk	功能
1	X	X	↑	同步清0
0	1	X		同步置数
0	0	1		计数（BCD）
0	0	0		保持

进位：当状态qout=59且cin=1时，cout=1，否则cout=0.

（3） BCD码计数规则：

- 状态=59，下一状态为0
- 状态≠59，个位=9时，下一状态：个位为0，十位加1
- 状态≠59，个位≠9 时，下一状态：个位加1，十位保持

（4）程序清单

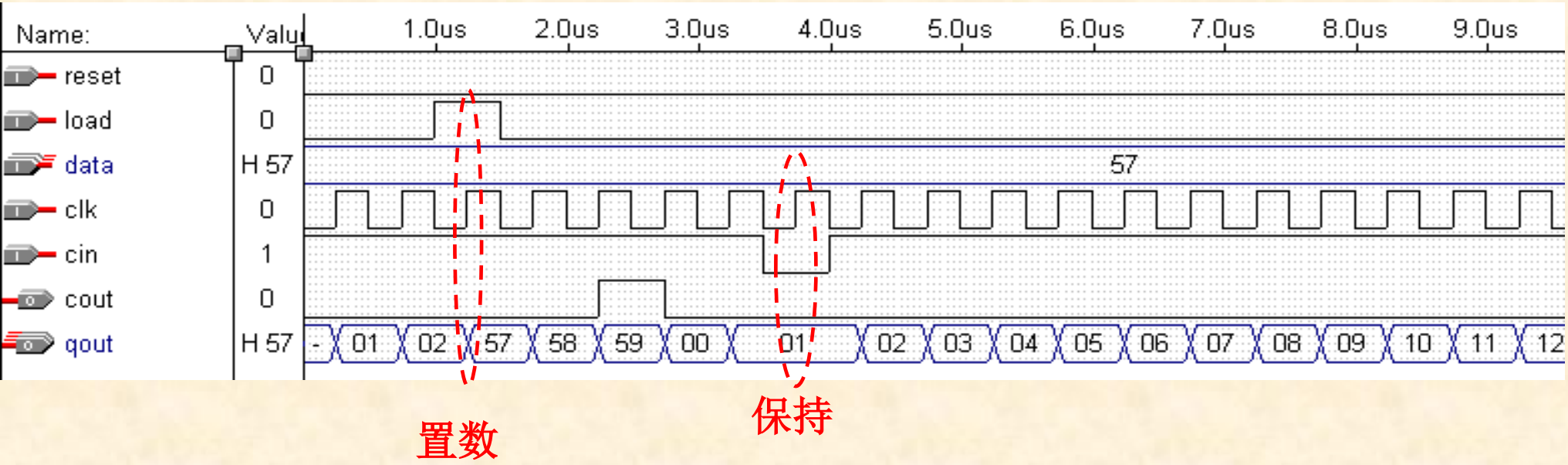
```
module count60(qout,cout,data,load,cin,reset,clk);
output[7:0] qout;
output  cout;
input[7:0] data;
input   load,cin,reset,clk;
reg [7:0]  qout;
assign  cout=(qout==8'h59)&cin;//进位
always @(posedge clk)
begin
 if(reset) qout=0;  //同步复位
 else  if    (load) qout=data;//同步置数
       else  if(cin)  //cin=1,计数    cin=0,保持
                 begin
                     if   (qout==8'h59)  qout=0 ; //qout=59,置0
                     else   if (qout[3:0]==9)     // 个位为9，个位置0，十位+1
                                begin qout[3:0]=0;qout[7:4]=qout[7:4]+1; end
                            // 个位不为9，个位+1,十位不变
                            else qout[3:0]<=qout[3:0]+1;
                 end
end
endmodule
```

reset	load	cin	clk	功能
1	X	X		同步清0
0	1	X		同步置数
0	0	1		计数（BCD）
0	0	1		保持

语句块

语句块

（5）仿真：



(6) 程序的改进：

通过定义常量，增加程序的通用性。

```
module count60(qout,cout,data,load,cin,reset,clk);
parameter  MODULUS=8'h59;
output[7:0] qout;
output  cout;
input[7:0] data;
input   load,cin,reset,clk;
reg [7:0]  qout;
assign  cout=(qout==MODULUS)&cin;//进位
always @(posedge clk)
      begin
        if(reset) qout=0;
        else   if(load) qout=data;//同步置数
               else   if(cin)  //cin=1,计数    cin=0,保持
                 begin
                        if(qout==MODULUS)  qout=0 ;
                        else  if (qout[3:0]==9)
                                 begin qout[3:0]=0;qout[7:4]=qout[7:4]+1; end
                              else qout[3:0]<=qout[3:0]+1;
                    end
      end
endmodule
```

改变MODULUS的值（<=99），即可改变模

二、任意进制计数器（带异步清0）

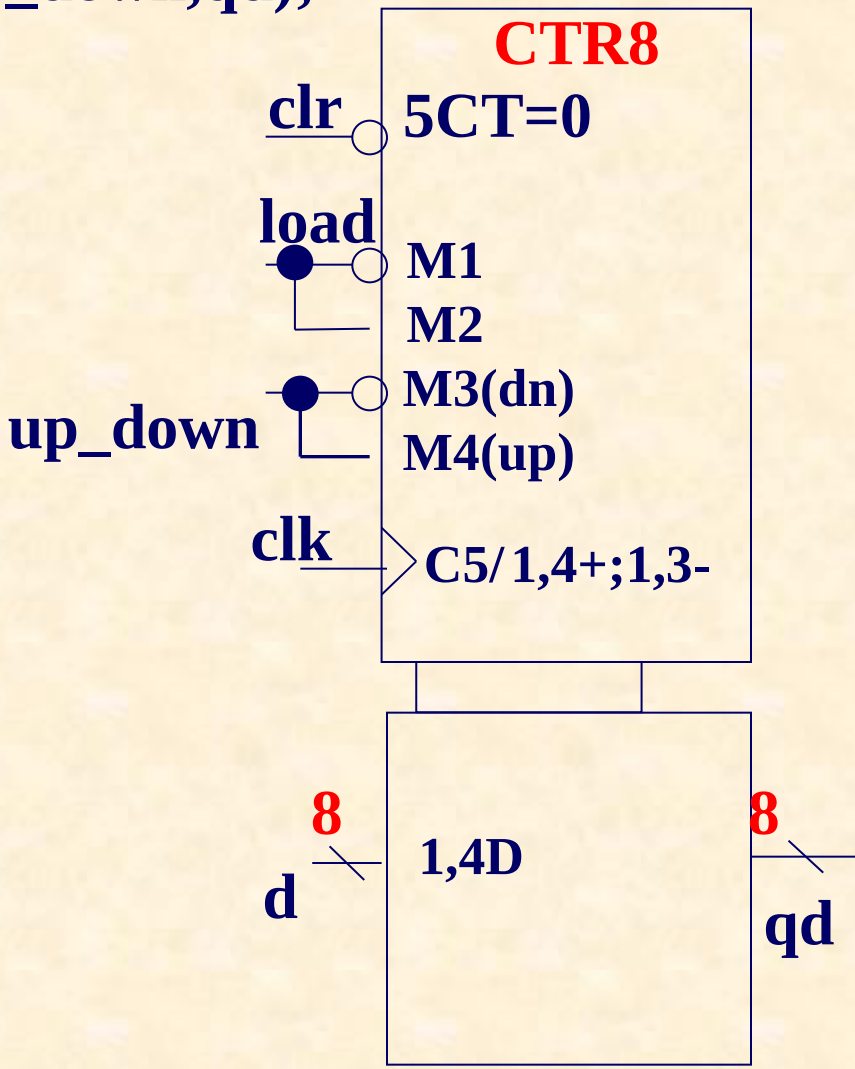
n:模； size： 计数器位数

```
module n_counter (Q, CO, EN ,CLR, CLK);
  parameter   n=30;
  parameter   size=5;
  output [size:1]  Q;
  output           CO;
  input            EN ,CLR, CLK;
   reg     [size:1]  Q;
  always @(posedge CLK or  negedge CLR) //异步清0
    begin
       if(!CLR)   Q=0;
       else  if(Q==(n-1))   Q=EN?0:Q; //计数或保持
             else Q = Q + EN;  //计数或保持
    end
   assign CO=(Q==(n-1))&EN; // 进位
endmodule
```

三、可预置的加减计数器

例：12.20

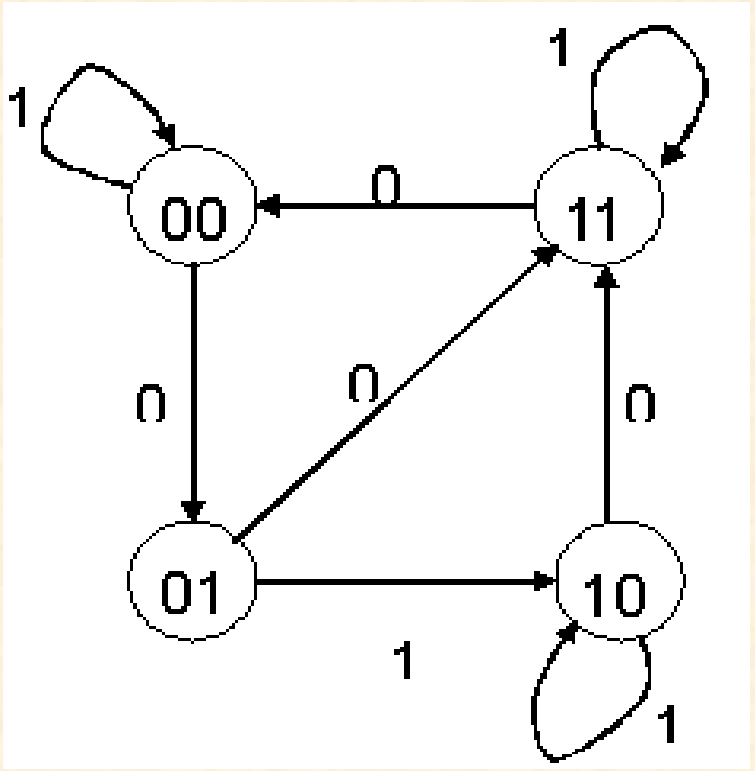
```
module up_down_count(d,clk,clear,load,up_down,qd);
parameter size=8;
input[size:1]   d;
input          clk,clear,load,up_down ;
output[size:1]  qd;
reg[size:1]     cnt;
assign qd=cnt;
always @(posedge clk)
begin
  if(!clear) cnt=0;//低电平、同步复位
  else if(load) cnt=d;//高电平、同步置数
      else if(up_down) cnt=cnt+1;//加法计数
          else cnt=cnt-1;//减法计数
end
endmodule
```



有限状态机的设计实例

状态机通常可使用带有always语句的case语句建模。状态信息存储在寄存器中。case语句的多个分支包含每个状态的行为。。

例　状态机的Verilog HDL的描述



```
module Moore_mdl (x, AB, CLK, RST);
    input x, CLK, RST;
    output [1:0] AB;
    reg [1:0] state;
    parameter      S0 = 2'b00, S1 = 2'b01,
    S2 = 2'b10, S3 = 2'b11;
      always @ (posedge CLK or negedge RST)
            if (!RST) state = S0;
            else
            case (state)
               S0: if (!x) state = S1; else state = S0;
               S1: if (x)   state = S2; else state = S3;
               S2: if (!x) state = S3; else state = S2;
               S3: if (!x) state = S0; else state = S3;
            endcase
    assign AB = state;
endmodule
```

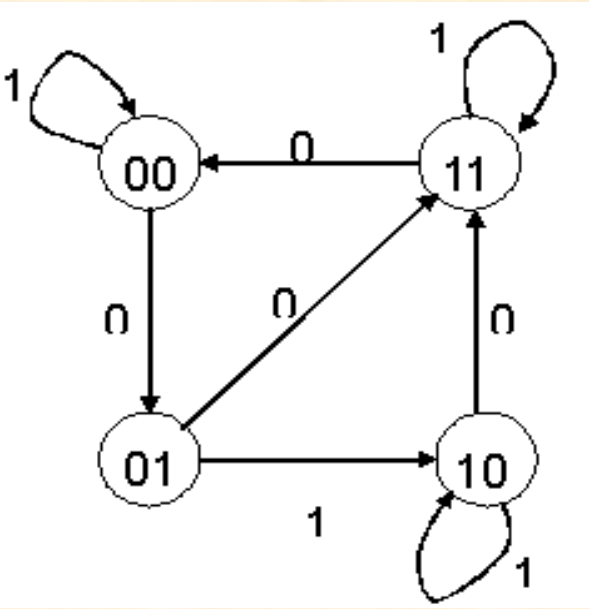
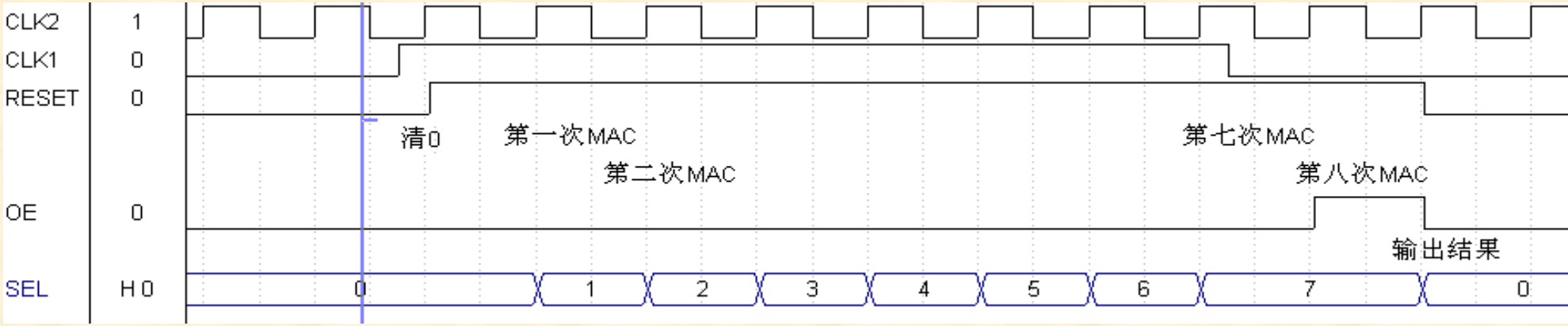


图7.11状态机图

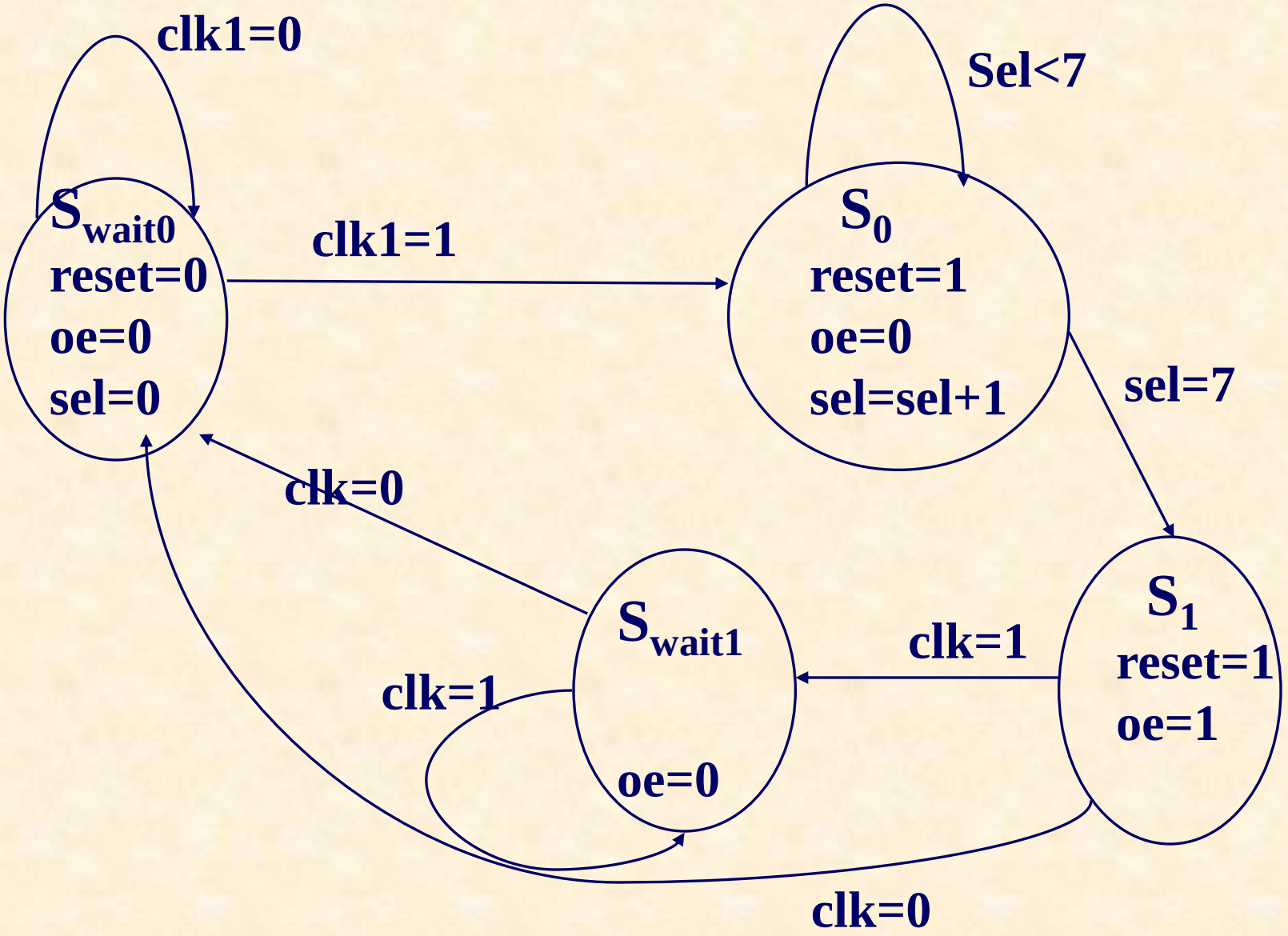
类似表状态

四、控制器的设计实例

某控制器时序要求如下图： CLK2为系统时钟, 每当输入信号出CLK1从低电平变为高电平时启动一次工作,即输出如下图的RESET、OE、SEL三个信号



•根据要求画出状态机图：


clk1=0
Sel<7
S wait0
reset=0
oe=0
sel=0
clk1=1
S 0
reset=1
oe=0
sel=sel+1
sel=7
clk=0
S wait1
oe=0
clk=1
clk=1
S 1
reset=1
oe=1
clk=0

•HDL代码：

注意:编码时将sel放在状态位中，　　f(clk2)>=10f(clk1)

```
module fir_control (clk1,clk2,clrn,reset,oe,sel);
        input clk1,clk2,clrn;
        output [2:0] sel;
    output reset,oe;
        reg [1:0] state;
    reg [2:0] sel;
    reg  reset,oe;
        parameter      Swait0=2'b00, S0 =2'b01,S1 =2'b10,
Swait1=2'b11;
```

```
always @ (posedge clk2 or negedge clrn)
        if (!clrn)  begin state = Swait0;oe=0;sel=0;reset=0;end
        else        case (state)
        Swait0:
         if (!clk1) begin oe=0;sel=0;reset=0;end
         else begin state=S0;oe=0;reset=1;end
           S0: if (sel<7)  begin sel=sel+1;reset=1;oe=0;end
         else begin  state = S1;reset=1;oe=1;end
           S1: if (clk1) begin state=Swait1;oe=0 ; end
         else begin state = Swait0;oe=0;sel=0;reset=0;end
           Swait0: if (!clk1) begin  state=Swait0;oe=0;sel=0;reset=0;end
        default :begin  state=Swait0;oe=0;sel=0;reset=0;end
    endcase
endmodule
```

课堂小结： Verilog HDL基础知识

- 概述
- Verilog HDL的基本结构
- 数据类型及常量、变量
- 运算符
- 语句
- 语句的顺序执行与并行执行
- 用Verilog HDL设计数字电路